
Курс: Рекомендательные системы

Занятие 3: Матричная факторизация

Подготовил: Лашинин Олег

Дата: 28.01.26

План занятия

- Проблема у i2i моделей с масштабированием
- Про характерные размеры матриц взаимодействий
- Про матричную факторизацию
- Классический вариант implicit ALS
- Развитие EASE через матричную факторизацию
- Explicit ALS
- BPR, SVD
- Про быстрый инференс векторных моделей
- Neural MF

Сложно ли масштабировать EASE?

USERS * ITEMS

1				1				
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	

X

ITEMS * ITEMS

0	.	.		.	.	.	.	.
.	0	.		.	.	.	.	.
.	.	0		.	.	.	.	.
.	.	.		.	.	.	.	.
.	.	.		.	.	.	.	.
.	.	.		.	0	.	.	.
.	.	.		.	.	0	.	.
.	.	.		.	.	.	0	.
.	.	.		.	.	.	.	0

=

USERS * ITEMS

1	.	.		.	.	.	.	.
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	



Что может быть сложно отмасштабировать?

Сложно ли масштабировать EASE?

USERS * ITEMS

1				1				
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	

X

ITEMS * ITEMS

0	.	.		.	.	.	.	.
.	0	.		.	.	.	.	.
.	.	0		.	.	.	.	.
.	.	.		.	.	.	.	.
.	.	.		.	.	.	.	.
.	.	.		.	0	.	.	.
.	.	.		.	.	0	.	.
.	.	.		.	.	.	0	.
.	.	.		.	.	.	.	0

=

USERS * ITEMS

1	.	.		.	.	.	.	.
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	



Что может быть сложно отмасштабировать?

- Item * item матрица тяжелая и долго считать
- Инференс: $M * M$ перемножений (M – число айтемов)

Характерные масштабы для РФ

Инференс: $M * M$ перемножений (M – число айтемов)



Где больше
и меньше
всего
юзеров и
айтемов?

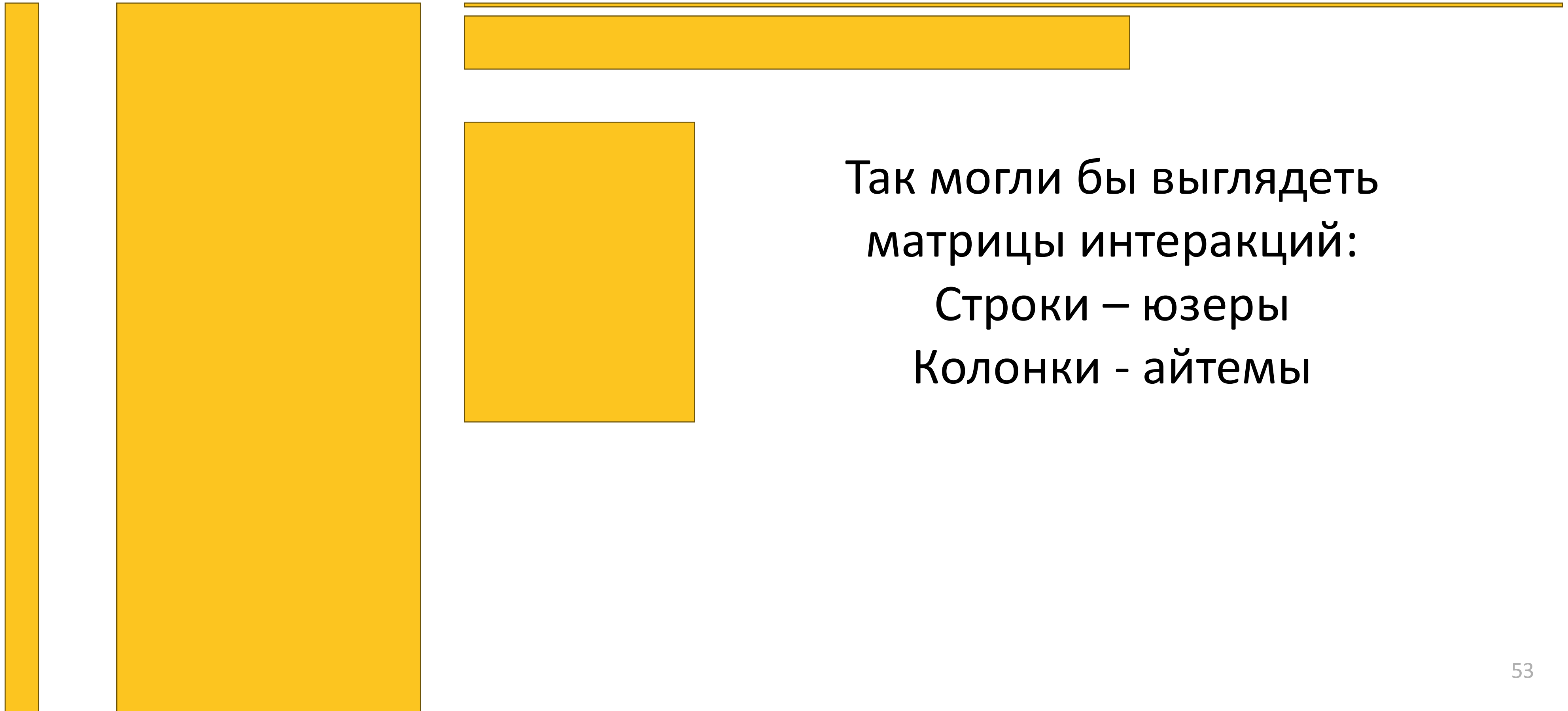
Домен	Число айтемов? (Порядок)	Число юзеров? (Порядок)
Музыка		
Посты в соц сети		
Видео на платформе		
Доставка продуктов		
Маркетплейсы		
Кофейни в Смоленске		
Категории повышенного кэшбека		
Квартиры у застройщика		
Банковские продукты		

Характерные масштабы для РФ

Не все проекты имеют много айтемов, но могут иметь десятки миллионов юзеров и приносить компании много денег!

Домен	Число айтемов? (Порядок)	Число юзеров? (Порядок)
Музыка	100M	10M
Посты в соц сети	1 млрд	100M
Видео на платформе	100м	10 M
Доставка продуктов	10к	10м
Маркетплейсы	100M	100M
Кофейни в Смоленске	100	10к
Категории повышенного кэшбека	100	10M
Квартиры у застройщика	1K	10к
Банковские продукты	100	100M

Характерные размеры матриц взаимодействий



Характерные размеры матриц взаимодействий

N – число юзеров, M число айтемов, тогда три интересных случая:

1) N большое, M – большое. Самый сложный случай, нужен сложный инжиниринг

2) N большое, но M – маленькое. Тогда:

- Вы можете использовать item2item модели
- Вы можете обучать item2item модели на подвыборке юзеров, но применять на всех юзерах
- Вы можете применять ItemKNN

3) N маленькое, но M – большое. Тогда:

- Вы можете использовать user2user модели, так как user'ов мало
- Скорее всего по большей части каталога вы не будете иметь данных

В проектах с маленьким M свои сложности!

Про матричную факторизацию

	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
User1	0.23	0.14	1			
User2	-0.4	0.2	0.1			
User3	1.5	2	3.6			
User4	0.2	-1.4	-0.24			
User5	-0.3	1	0.46			
User6	1.4	-0.3	0.6			
User7	-0.3	1	0.23			
User8	1.5	0.5	1.6			

X

Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
0.23	-2.4	2.3	-1	0.1	0.67
-0.42	-0.3	0.6	0.9	1.8	-0.3
0.4	0.1	-0.3	0.6	2.3	3.5

=

USERS * ITEMS

				1
	1			
	1			1
		1		
			1	
	1			

Матрица взаимодействий

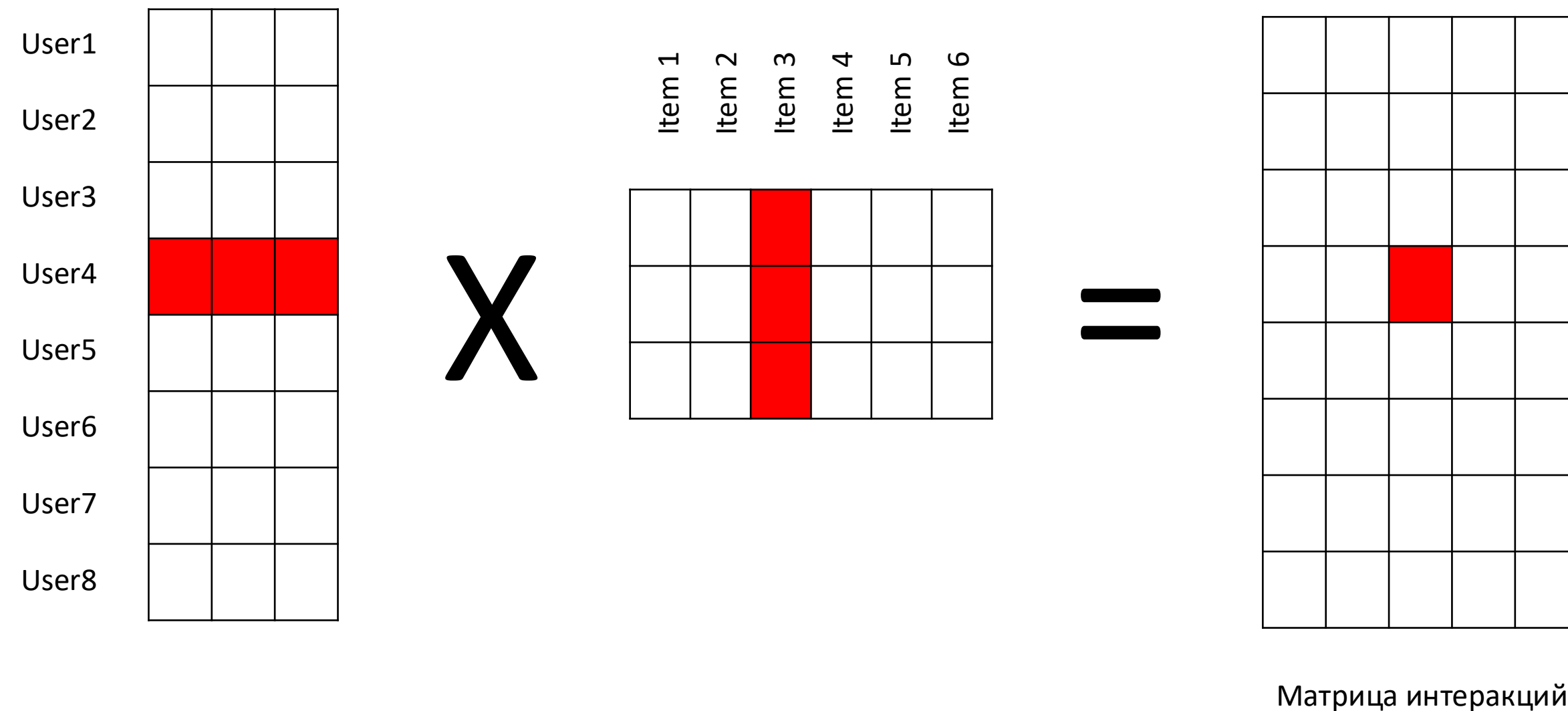
Перемножение
матриц:

Матрица **A**: $U * d$

Матрица **B**: $I * d$

Тогда **A * B^T** будет
иметь
размерность **U x I**

Про матричную факторизацию



Перемножение
матриц:

Матрица **A**: $U * d$

Матрица **B**: $I * d$

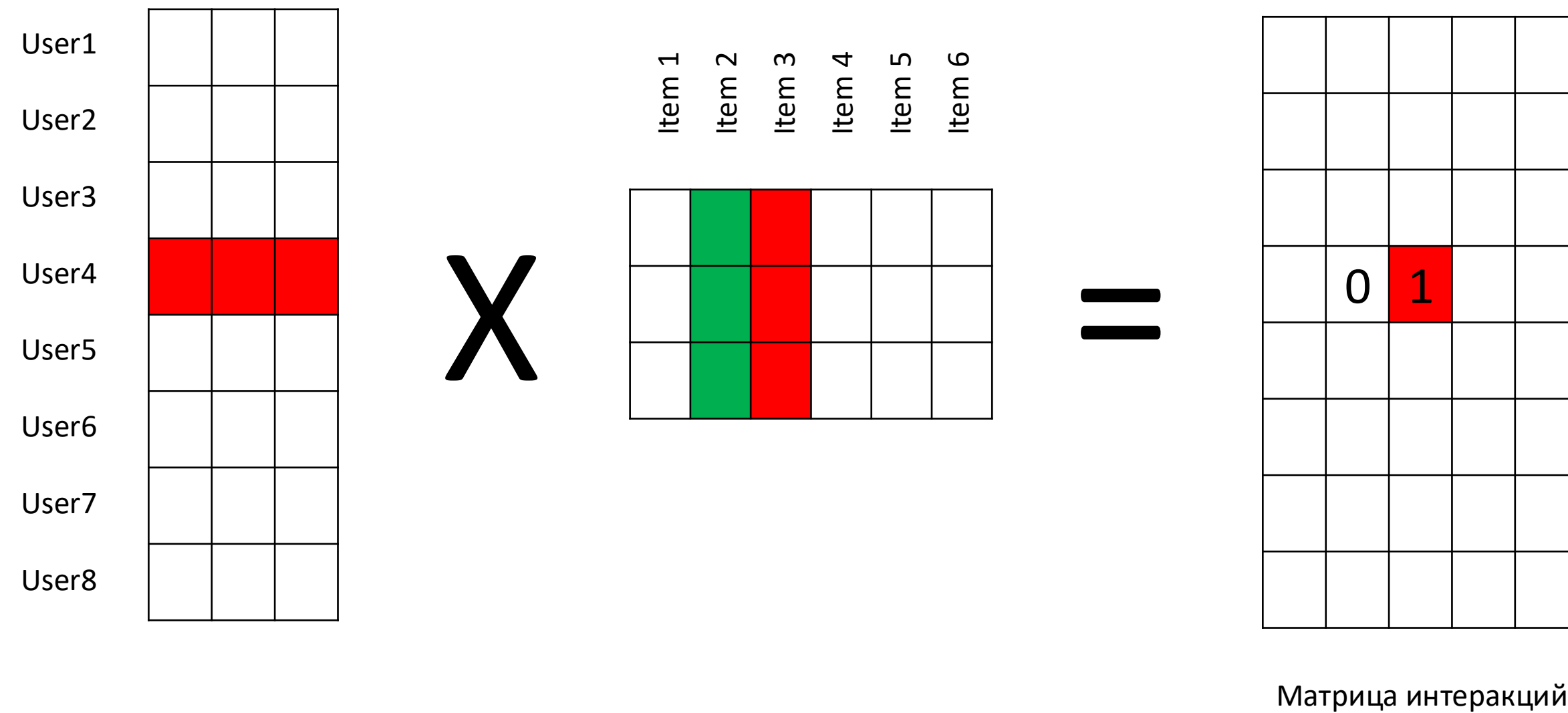
Тогда $A * B^T$ будет
иметь

размерность $U \times I$

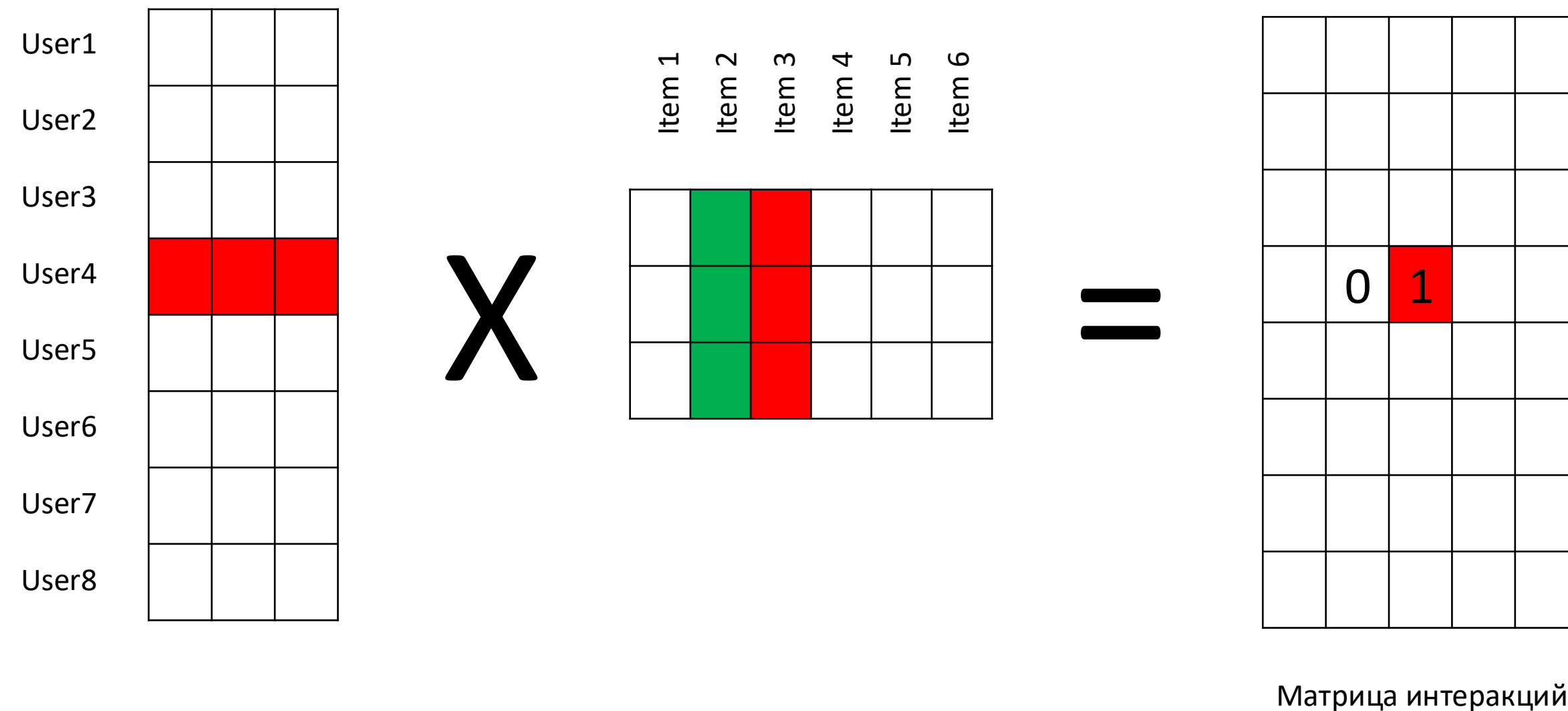
$$(X_1, X_2, X_3) - \text{вектор } u, (Y_1, Y_2, Y_3) - \text{вектор } i,$$

$$X_1 * Y_1 + X_2 * Y_2 + X_3 * Y_3 = r_{ui}$$

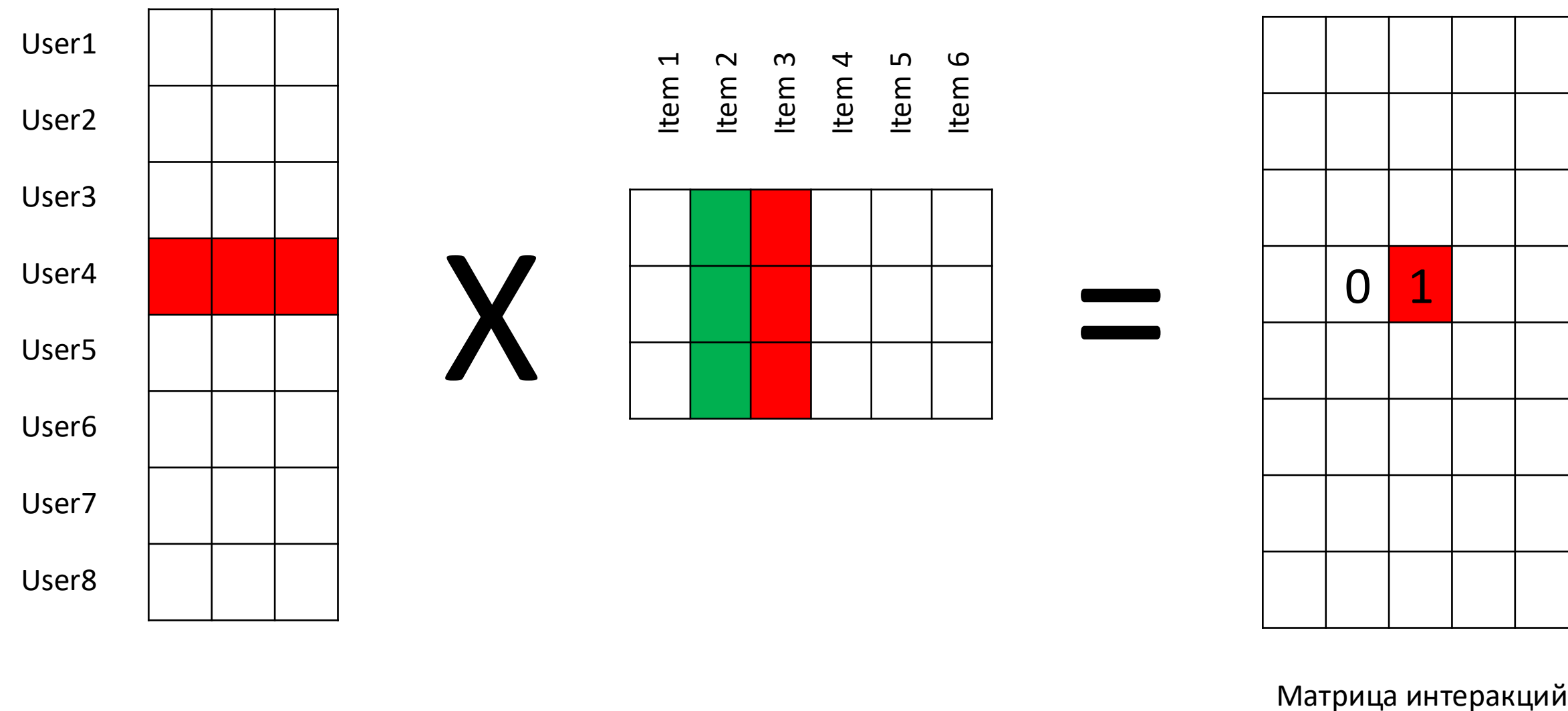
Про матричну факторизацію



Про матричную факторизацию



Про матричную факторизацию



Про матричную факторизацию

$$\begin{aligned} X_1^u * Y_1^i + X_2^u * Y_2^i + X_3^u * Y_3^i &= 1 \\ X_1^u * Y_1^j + X_2^u * Y_2^j + X_3^u * Y_3^j &= 0 \end{aligned}$$

$$L = \sum_{u,i} (r_{ui} - x^T y)^2 + \lambda(\|x\|^2 + \|y\|^2)$$



Как можно найти нужные вектора?

Матричная факторизация через SGD

1. Исходная задача (та же самая)

Мы минимизируем:

$$\mathcal{L} = \sum_{u,i} c_{ui} (p_{ui} - x_u^\top y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right)$$

Где:

- $p_{ui} \in \{0, 1\}$
- $c_{ui} \geq 1$

2. Стохастическая версия

SGD берёт одну пару (u, i) за шаг:

$$\ell_{ui} = c_{ui} (p_{ui} - x_u^\top y_i)^2 + \lambda (\|x_u\|^2 + \|y_i\|^2)$$

3. Ошибка и предсказание

$$\hat{p}_{ui} = x_u^\top y_i$$

$$e_{ui} = p_{ui} - \hat{p}_{ui}$$

4. Градиенты (строго)

Для одного примера:

$$\frac{\partial \ell_{ui}}{\partial x_u} = -2c_{ui}e_{ui}y_i + 2\lambda x_u$$

$$\frac{\partial \ell_{ui}}{\partial y_i} = -2c_{ui}e_{ui}x_u + 2\lambda y_i$$

5. SGD-обновления

С шагом обучения η :

$$\begin{aligned} x_u &\leftarrow x_u - \eta \frac{\partial \ell_{ui}}{\partial x_u} \\ &= x_u + \eta (c_{ui}e_{ui}y_i - \lambda x_u) \end{aligned}$$

$$y_i \leftarrow y_i + \eta (c_{ui}e_{ui}x_u - \lambda y_i)$$

Матричная факторизация через ALS

Alternating Least
Squares
(ALS)

[Collaborative filtering for implicit feedback datasets](#)

[Y Hu, Y Koren, C Volinsky - 2008](#) Eighth IEEE international ..., 2008 - [ieeexplore.ieee.org](#)

... **Alternating least squares** was used for explicit **feedback** ... **collaborative** filtering on datasets with **implicit feedback**, ... One of our main findings is that **implicit** user observations should be ...

☆ Save ↗ Cite Cited by 4681 Related articles All 17 versions

[\[PDF\] chrisvolinsky.com](#)

Explicit ALS

Обучается на
явном фидбеке
от
пользователей
(рейтингах)

Implicit ALS

Обучается на
неявном
фидбеке
(покупки, клики)

Основная идея ALS – поочередно фиксируем одну из матриц и явно выписываем оптимальное решение

Матричная факторизация через ALS

- Каждой паре user–item сопоставляется
 - preference** $p_{ui} \in \{0, 1\}$ (было взаимодействие или нет)
 - confidence** $c_{ui} = 1 + \alpha r_{ui}$ (насколько мы уверены в предпочтении)

Оптимизация iALS (Implicit ALS)

- Фиксируем эмбединги айтемов Y и независимо для каждого пользователя решаем задачу взвешенной ridge-регрессии:

$$x_u = \arg \min_x \sum_i c_{ui} (p_{ui} - x^\top y_i)^2 + \lambda \|x\|^2$$

- Решение имеет **закрытую форму** (нормальные уравнения):

$$x_u = (Y^\top C_u Y + \lambda I)^{-1} Y^\top C_u p_u$$

где $C_u = \text{diag}(c_{ui})$

- Аналогично обновляются эмбединги айтемов при фиксированных пользователях
- Ключевая оптимизация:
 - $C_u = I + \alpha R_u$, где R_u разреженная
 - Общая матрица $Y^\top Y$ считается **один раз**
 - Для каждого пользователя добавляется только поправка по его взаимодействиям
- Итог:
 - сложность $\mathcal{O}((|U| + |I|)k^3 + |R|k^2)$
 - легко параллелится
 - устойчив к сильному дисбалансу нулей

- k — размерность эмбедингов
- $|I|$ — количество айтемов
- $x_u \in \mathbb{R}^k$ — вектор пользователя
- $y_i \in \mathbb{R}^k$ — вектор айтема
- $Y \in \mathbb{R}^{|I| \times k}$ — матрица всех айтемов (каждая строка — айтем)
- $p_u \in \mathbb{R}^{|I|}$ — вектор предпочтений пользователя
- $C_u \in \mathbb{R}^{|I| \times |I|}$ — диагональная матрица confidence

Матричная факторизация через ALS

$$x_u = A_u^{-1} b_u$$

то есть

$$x_u = \left(Y^T Y + \alpha \sum_{i \in \mathcal{I}_u} y_i y_i^T + \lambda I \right)^{-1} \left((1 + \alpha) \sum_{i \in \mathcal{I}_u} y_i \right)$$

Это и есть обновление user-вектора в implicit ALS.

Откуда это берётся (одним логическим шагом)

Мы минимизируем по x_u :

$$L(x_u) = \sum_i c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda \|x_u\|^2$$

Берём градиент по x_u и приравниваем нулю:

$$-2 \sum_i c_{ui} y_i (p_{ui} - x_u^T y_i) + 2\lambda x_u = 0$$

Перегруппируем:

$$\left(\sum_i c_{ui} y_i y_i^T + \lambda I \right) x_u = \sum_i c_{ui} y_i p_{ui}$$

В матричном виде:

$$(Y^T C_u Y + \lambda I) x_u = Y^T C_u p_u$$

Отсюда:

$$x_u = (Y^T C_u Y + \lambda I)^{-1} Y^T C_u p_u$$

Ситуация, когда пришел новый пользователь

Используем **ALS-формулу** для одного пользователя:

$$x_{u_{\text{new}}} = (Y^T C_u Y + \lambda I)^{-1} Y^T C_u p_u$$

- $C_u = \text{diag}(c_{ui})$
- p_u — вектор предпочтений по айтемам
- $x_{u_{\text{new}}}$ — вектор эмбединга нового пользователя

Важно: суммирование идёт только по айтемам, с которыми есть взаимодействия, чтобы не считать всю матрицу $|I|$ целиком (разреженно).



Какой еще кейс можно покрыть пересчетом векторов?

Ситуация, когда пришел новый пользователь

Используем **ALS-формулу** для одного пользователя:

$$x_{u_{\text{new}}} = (Y^T C_u Y + \lambda I)^{-1} Y^T C_u p_u$$

- $C_u = \text{diag}(c_{ui})$
- p_u — вектор предпочтений по айтемам
- $x_{u_{\text{new}}}$ — вектор эмбединга нового пользователя

Важно: суммирование идёт только по айтемам, с которыми есть взаимодействия, чтобы не считать всю матрицу $|I|$ целиком (разреженно).



Какой еще кейс можно покрыть пересчетом векторов?

Когда юзер сделал новые действия

Ситуация, когда пришел новый айтем

$$y_{i_{\text{new}}} = (X^{\top} C_i X + \lambda I)^{-1} X^{\top} C_i p_i$$

Можно получить вектор айтема через юзеров, которые с ним взаимодействовали

Пайплайн обновления модели

Дорого и долго, но качественно

Большое
обновление
модели
полностью

Большое
обновление
модели
полностью

Большое
обновление
модели
полностью

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Маленькие
апдейты

Быстро, но менее качественно

→ время

Что такое confidence?

- Каждой паре user–item сопоставляется
 - **preference** $p_{ui} \in \{0, 1\}$ (было взаимодействие или нет)
 - **confidence** $c_{ui} = 1 + \alpha r_{ui}$ (насколько мы уверены в предпочтении)

Вы можете брать:

- Просто бинарные действия (например, клики)
- Вы можете взять, например, время просмотра фильмов
- Либо сделать сумму $w1 * \text{число покупок} + w2 * \text{число кликов} + w3 * \text{число лайков}$

Здесь нет универсального рецепта

Объяснение рекомендаций

$$W_u = (Y^\top C_u Y + \lambda I)^{-1} \in \mathbb{R}^{f \times f}$$

- f — число латентных факторов
- $Y \in \mathbb{R}^{|I| \times f}$ — матрица эмбедингов айтемов
- $C_u \in \mathbb{R}^{|I| \times |I|}$ — диагональная матрица confidence для пользователя u
- λI — регуляризация

iALS: объяснение рекомендаций

- Каждому пользователю u сопоставляется **весовая матрица** W_u , отражающая важность прошлых взаимодействий.
- **Взвешенная схожесть** между айтемами i и j с точки зрения u :

$$s_{ij}^u = y_i^\top W_u y_j$$

- Предсказанное предпочтение для айтема i :

$$\hat{p}_{ui} = \sum_{j:r_{uj}>0} s_{ij}^u c_{uj}$$

- Интерпретация:
 - Модель становится **линейной по прошлым действиям**
 - Каждое прошлое взаимодействие вносит **отдельный вклад** в предсказание
 - Вклад можно разложить на две части:
 1. **Важность для пользователя** — c_{uj}
 2. **Сходство с целевым айтемом** — s_{ij}^u
- Действия с наибольшим вкладом становятся **основным объяснением рекомендации**

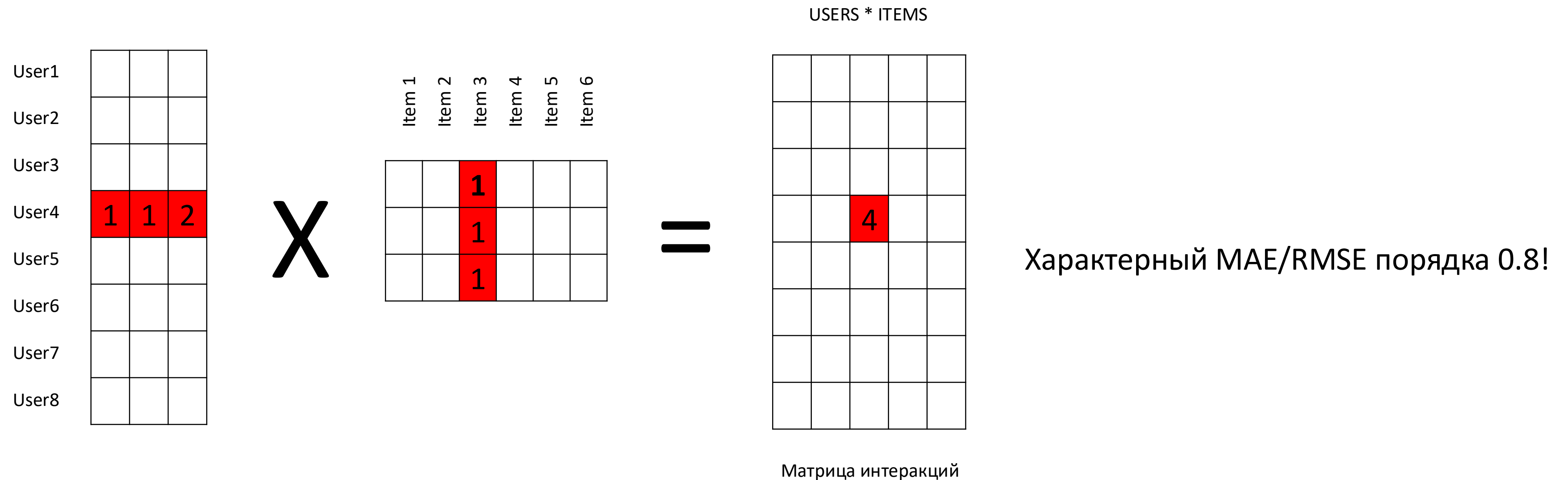
Например, можно сказать
 пользователю, мы тебе
 рекомендуем X,
 Потому что ты
 взаимодействовал с
 1) Y, вот с такой частотой
 2) Z, с другой частотой

И т. Д.

Про важные гиперпараметры

- 1) Размерность эмбеддингов
- 2) Alpha – коэффициент при наших событиях
- 3) Lambda – степень регуляризации
- 4) Число итераций

Про Explicit ALS



Ваш рейтинг фильма = dot product (ваш вектор, вектор фильма)

$$L = \sum_{u,i} (r_{ui} - x^T y)^2 + \lambda (\|x\|^2 + \|y\|^2)$$

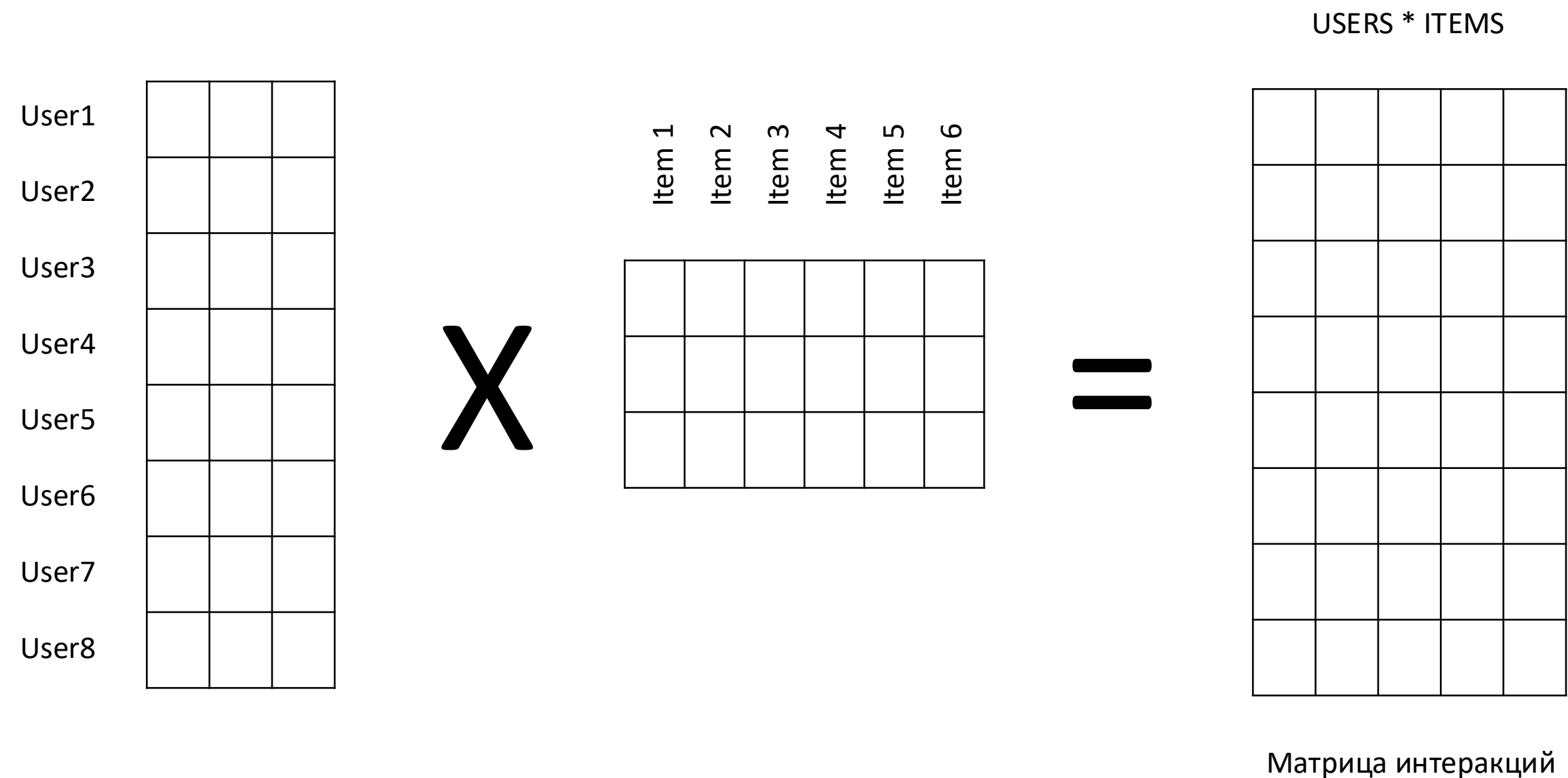
Про рейтинги

Несколько утверждений:

- 1) Рейтинги редко встречаются, в основном в курсах по recsys 😊. Пользователи не любят их оставлять
- 2) Максимизация рейтинга может идти вразрез бизнес метрикам. Например, вам нравится продукт, но если компания вам его продает, она будет в минусе
- 3) Если у вас есть 2 модели, одна на рейтинг, другая на implicit действия, скорее всего вторая будет нужнее для максимизации действий. Почему?



Про быстрый поиск рекомендаций

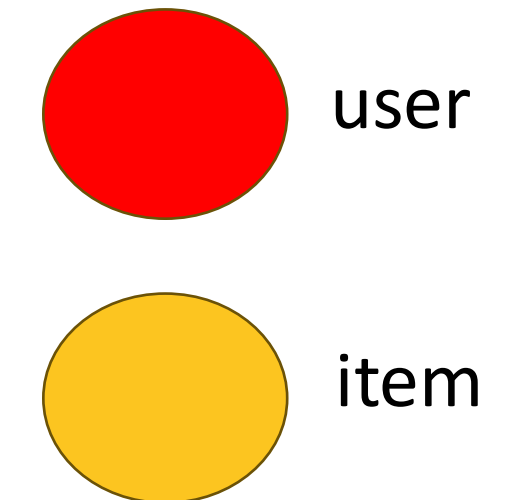
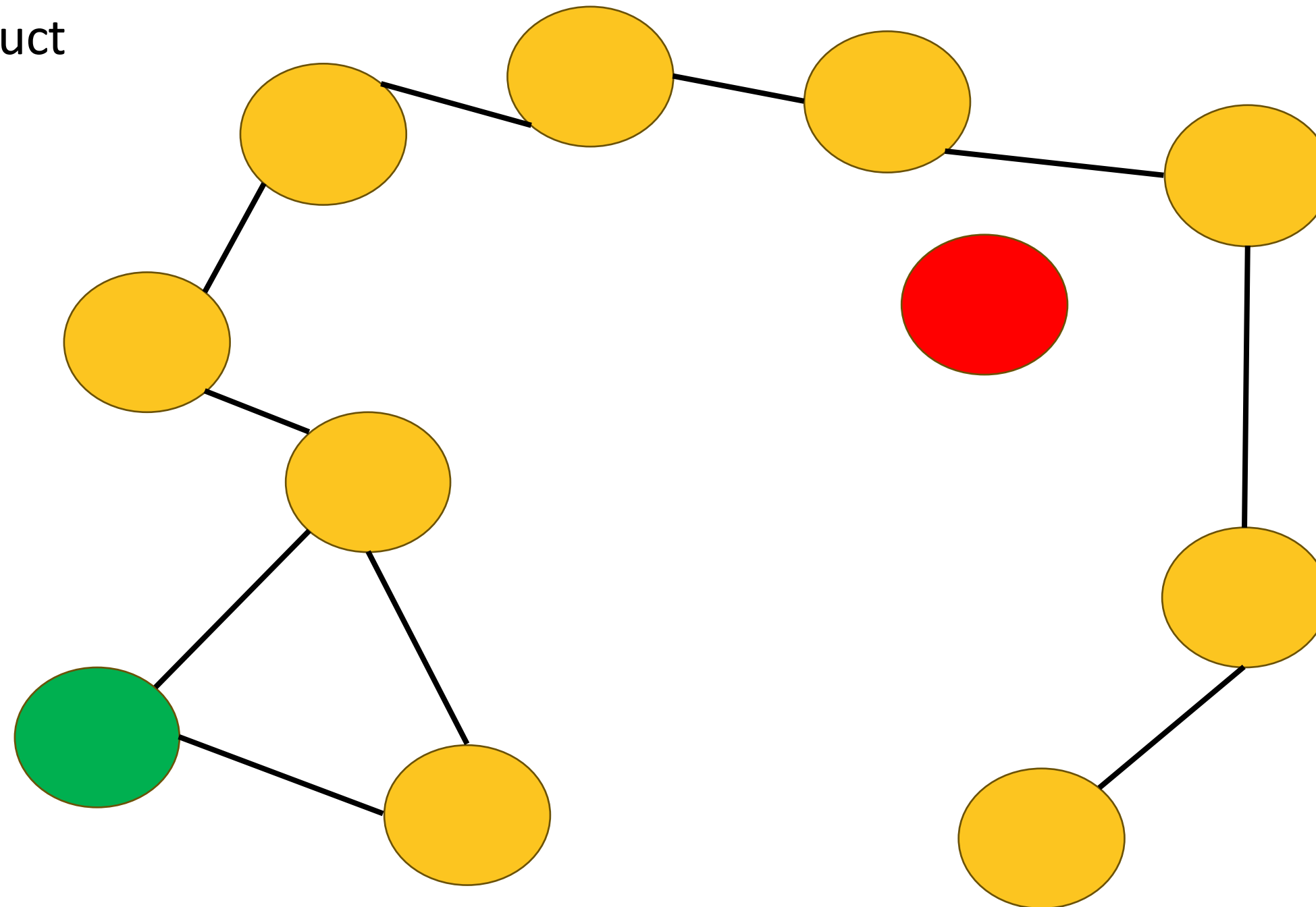


- Если d – размерность эмбедингов, M – число айтемов, тогда
- 1) Делаем $M * d$ перемножений
 - 2) Получаем список длины M
 - 3) Сортируем $M * \log M$ список (либо берем topk, тогда $O(m + k \log k)$)

Approximate Nearest Neighbor (ANN) search

Пространство векторов user/item

Близость – dot product

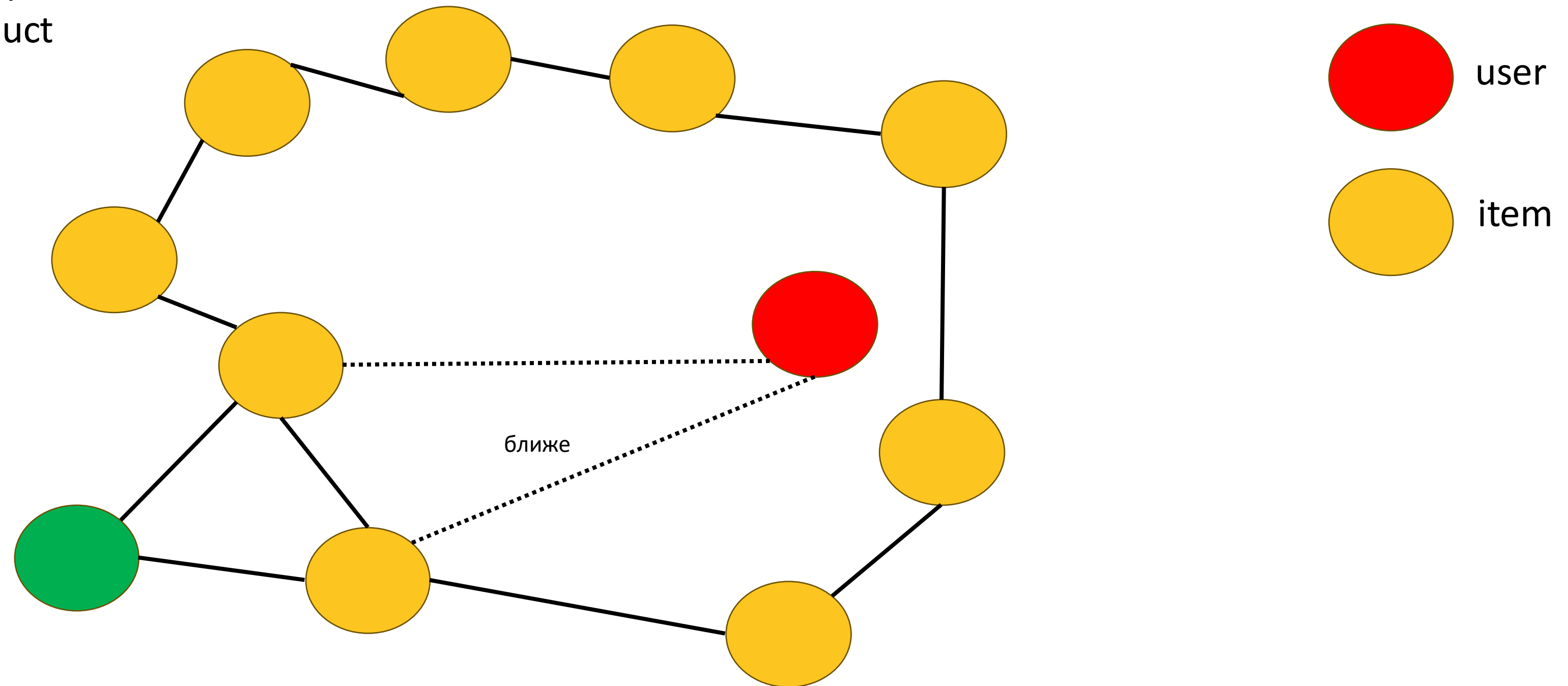


Стартовая точка поиска рекомендаций

Approximate Nearest Neighbor (ANN) search

Пространство векторов user/item

Близость – dot product

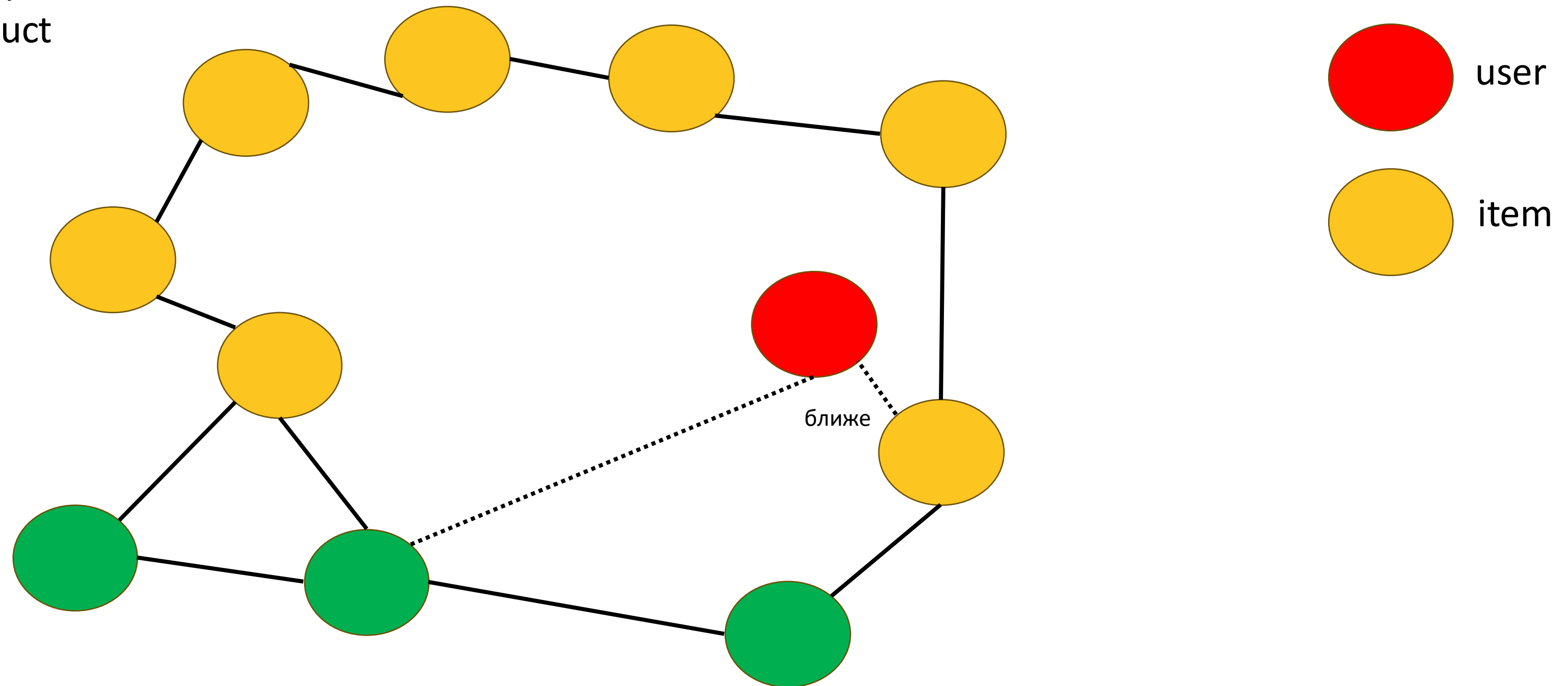


Стартовая точка поиска рекомендаций

Approximate Nearest Neighbor (ANN) search

Пространство векторов user/item

Близость – dot product

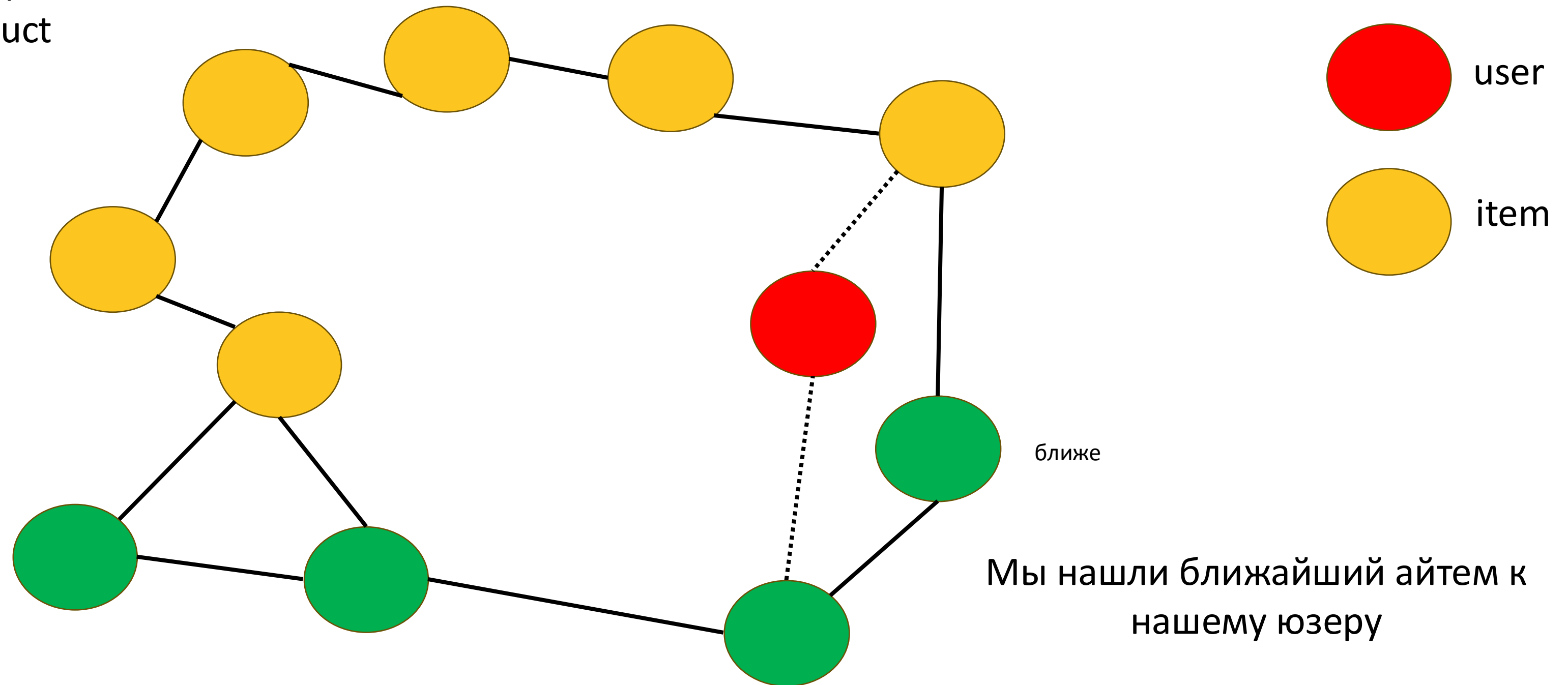


Стартовая точка поиска рекомендаций

Approximate Nearest Neighbor (ANN) search

Пространство векторов user/item

Близость – dot product

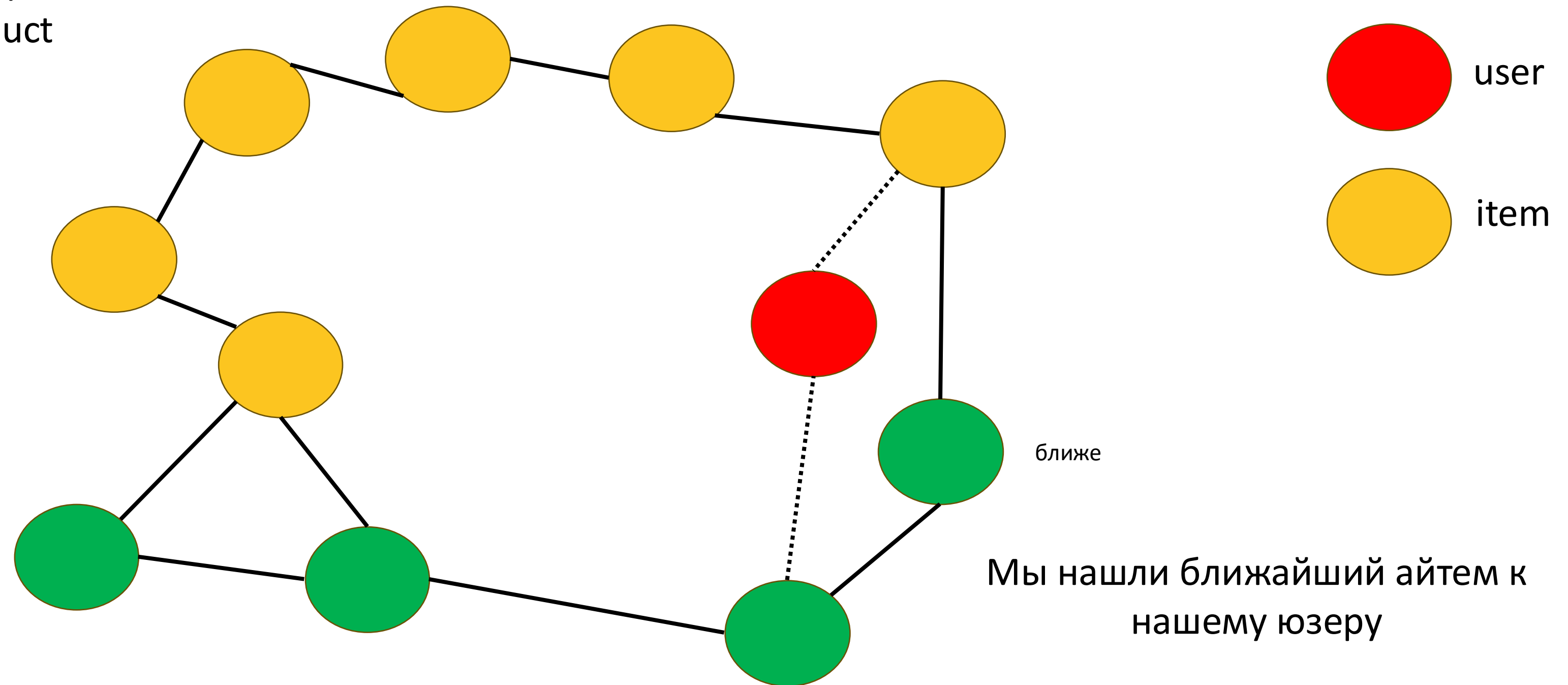


Стартовая точка поиска рекомендаций

Approximate Nearest Neighbor (ANN) search

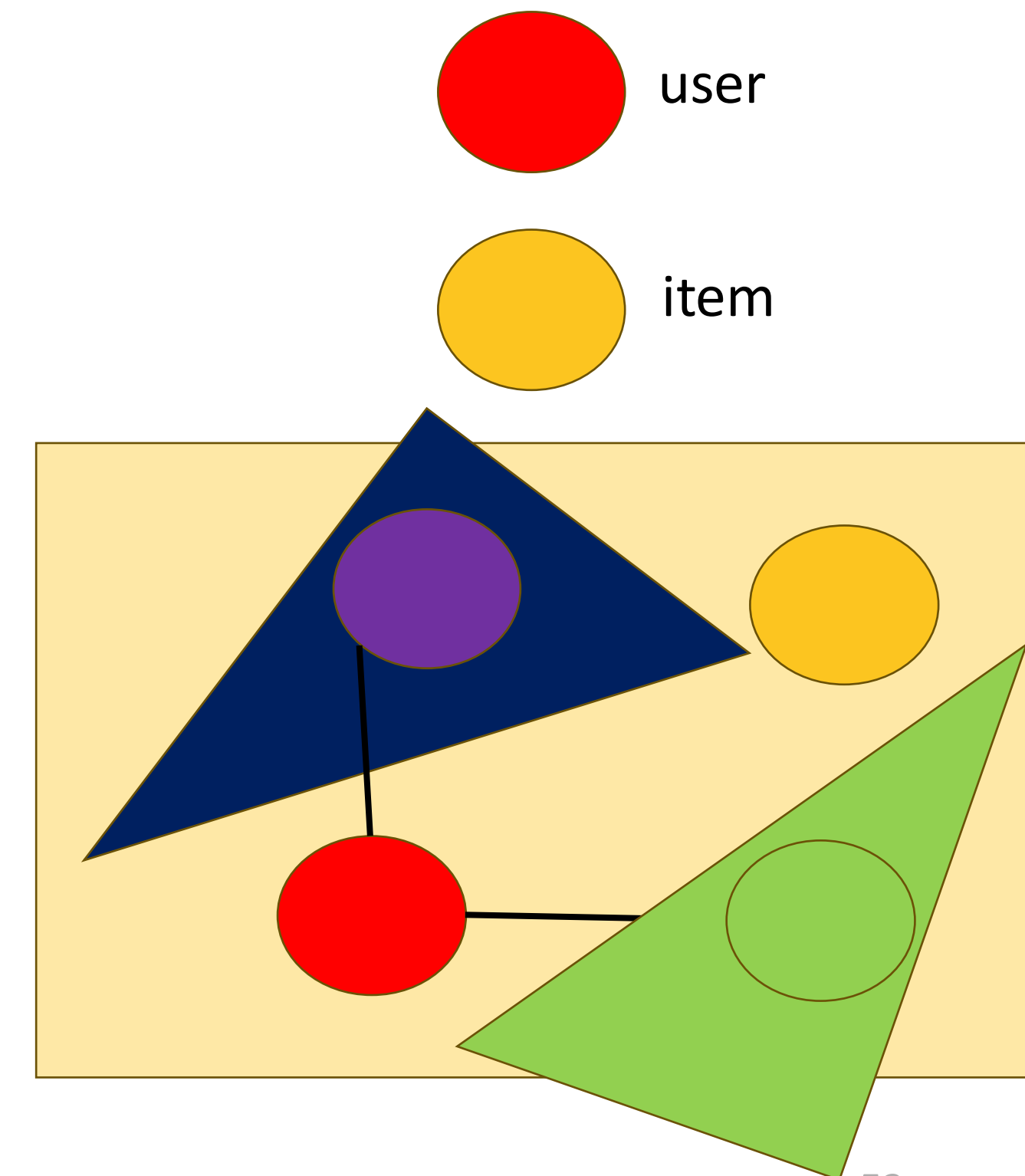
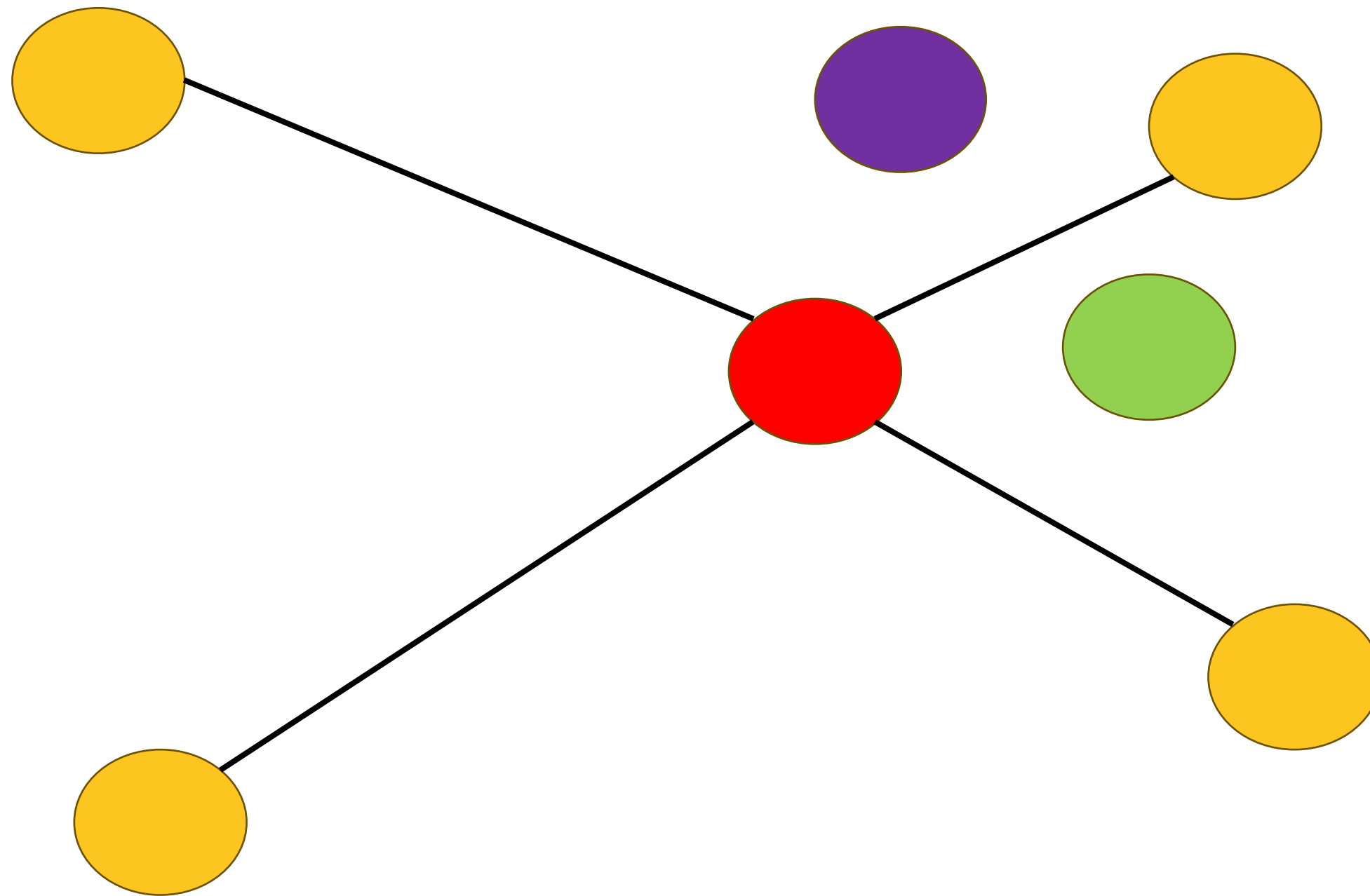
Пространство векторов user/item

Близость – dot product



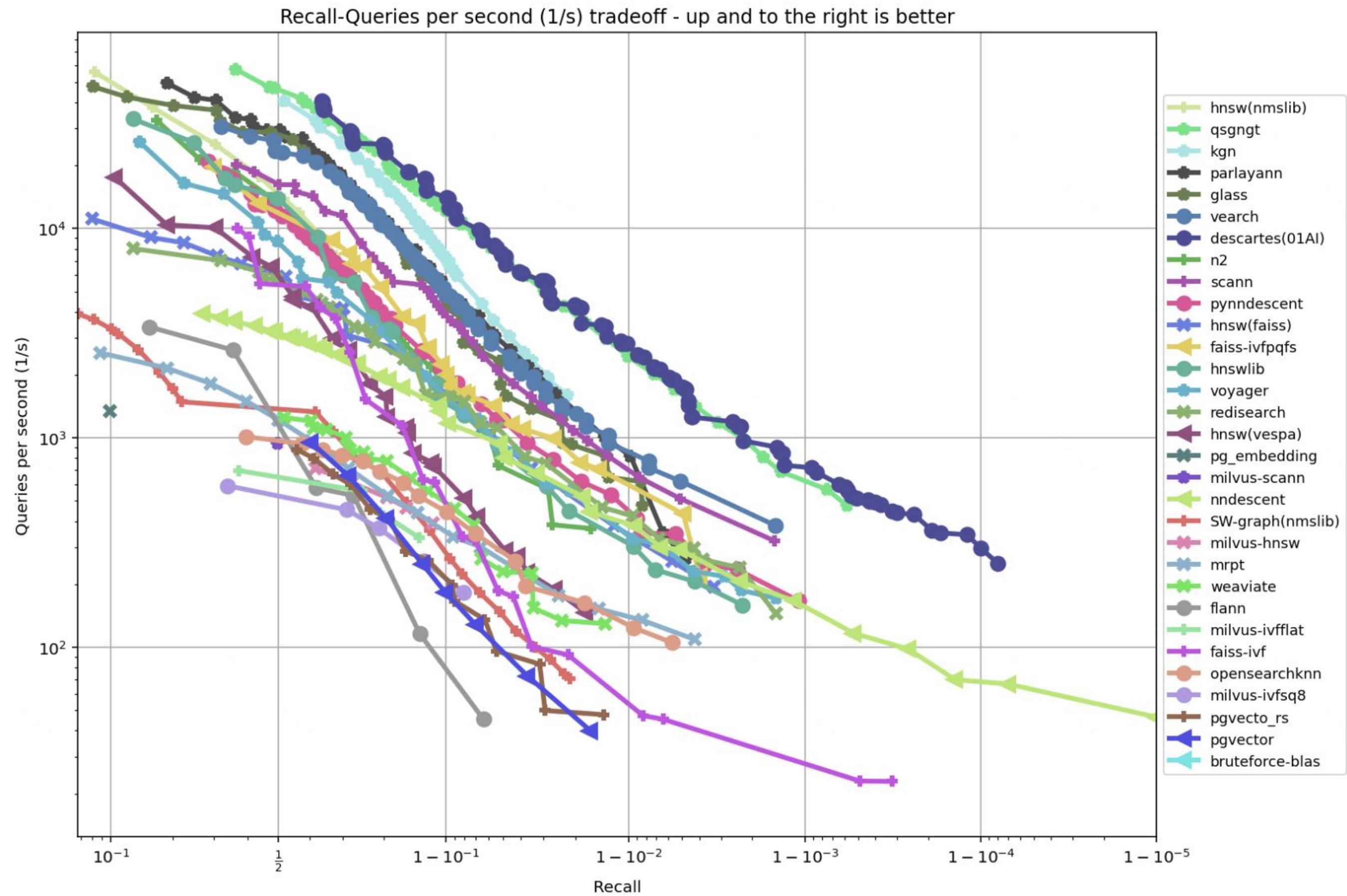
Стартовая точка поиска рекомендаций

Еще одна интуиция про ANN



Такой алгоритм ищет за $O(\log N)$, тогда как полный перебор $O(N)$

Бенчмарк



Разница между прямым и ANN способом

	Прямой способ	ANN
Сложность по времени	$Md + k \log M$	$K \log M$
Ограничения?	Работает всегда	Работает только если в конце модель оптимизировала user * item эмбединг, <u>не только для MF!</u>
Что надо хранить?	User * num_recs записей	User * d + Item * d значений
Масштабируется по M?	Очень плохо	Хорошо
Зависит ли хранение от длины рекомендаций?	Да	Нет
Гарантирует ли 100% качества от возможного?	Да	Нет
Работает ли на i2i моделях (там где i2i матрица)	Да	Скорее нет, но может быть

Выбор между iALS и EASE

Revisiting the Performance of iALS on Item Recommendation Benchmarks

Steffen Rendle
Google
Mountain View, USA
srendle@google.com

Li Zhang
Google
Mountain View, USA
liqzhang@google.com

Walid Krichene
Google
Mountain View, USA
walidk@google.com

Yehuda Koren
Google
Haifa, Israel
yehuda@google.com

Table 2: Quality results on the ML20M and MSD benchmark sorted by Recall@20 scores.

Dataset	Method	Recall@20	Recall@50	NDCG@100	Result from
ML20M	RecVAE [25]	0.414	0.553	0.442	[25]
	H+Vamp (Gated) [14]	0.413	0.551	0.445	[14]
	RaCT [18]	0.403	0.543	0.434	[18]
	Mult-VAE [17]	0.395	0.537	0.426	[17]
	LambdaNet [4]	0.395	0.534	0.427	[25]
	iALS	0.395	0.532	0.425	our result
	EASE [26]	0.391	0.521	0.420	[26]
	CDAE [28]	0.391	0.523	0.418	[17]
	Mult-DAE [17]	0.387	0.524	0.419	[17]
	SLIM [19]	0.370	0.495	0.401	[17]
	iALS	0.363	0.502	0.393	prev. result [13]
	iALS	0.360	0.498	0.386	prev. result [17]
	WARP [27]	0.314	0.466	0.341	[25]
	Popularity	0.162	0.235	0.191	[26]
MSD	EASE [26]	0.333	0.428	0.389	[26]
	iALS	0.309	0.415	0.368	our result
	RecVAE [25]	0.276	0.374	0.326	[25]
	RaCT [18]	0.268	0.364	0.319	[18]
	Mult-VAE [17]	0.266	0.364	0.316	[17]
	Mult-DAE [17]	0.266	0.363	0.313	[17]
	LambdaNet [4]	0.259	0.355	0.308	[25]
	iALS	0.211	0.312	0.257	prev. result [17]
	WARP [27]	0.206	0.302	0.249	[25]
	CDAE [28]	0.188	0.283	0.237	[17]
	Popularity	0.043	0.068	0.058	[26]

Но если айтемов много, EASE не работает, iALS работает

Развитие EASE для масштабируемости

RecSys '22, September 18–23, 2022, Seattle, WA, USA

Vojtěch Vančura, Rodrigo Alves, Petr Kasalický, and Pavel Kordík

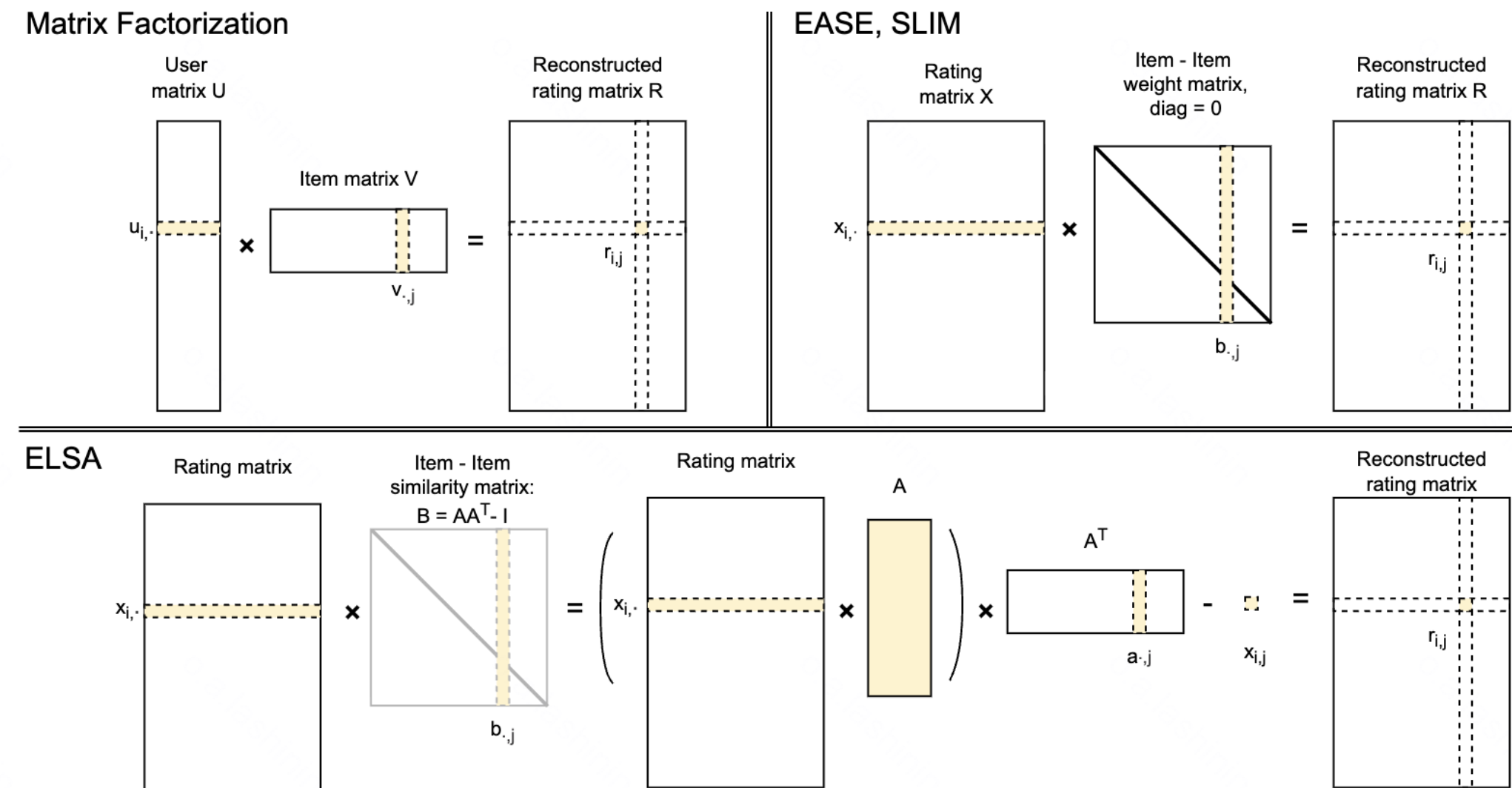


Figure 1: Differences between matrix factorization, traditional shallow autoencoders and ELSA.

Optimization problem: ELSA regards to the following optimization:

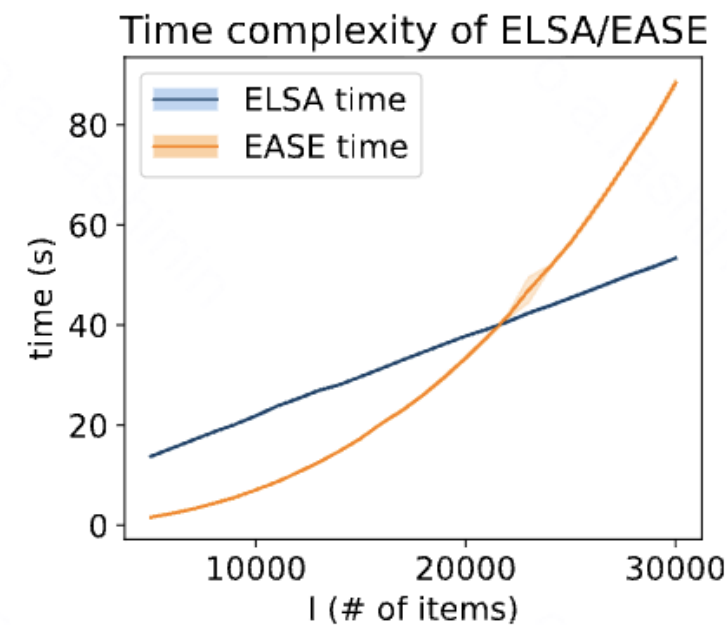
$$\min_A \|f(X) - f(X(AA^T - I))\|_F^2 \quad \text{s.t.} \quad \|A_{a,\cdot}\|^2 = 1 \quad (1)$$

where $A \in \mathbb{R}^{I \times r}$ is a latent matrix, I is the identity matrix and $r \ll I$ is a (tunable) hyperparameter that reduces the latent space and helps to avoid over-fitting. ELSA draws inspiration from the cosine

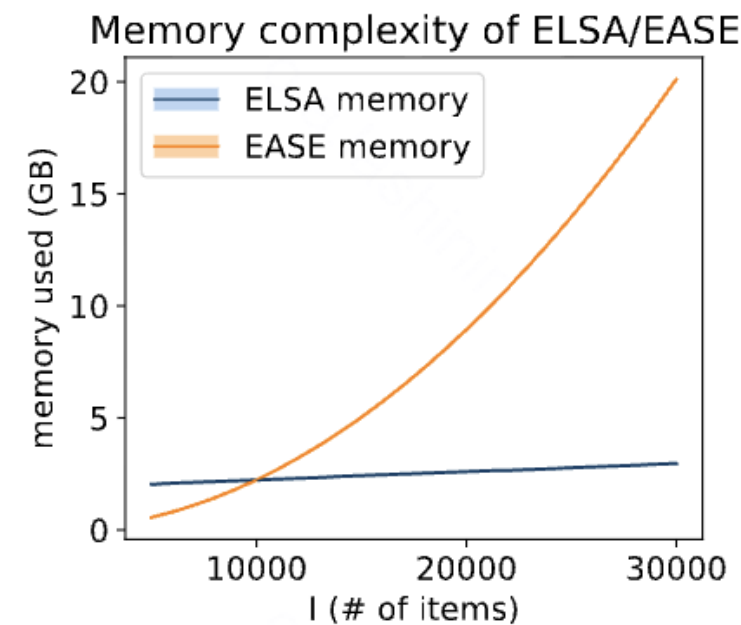
Развитие EASE для масштабируемости

Table 1: Overall results. $N@k$ stands for NDCG@k, $R@k$ for Recall@k

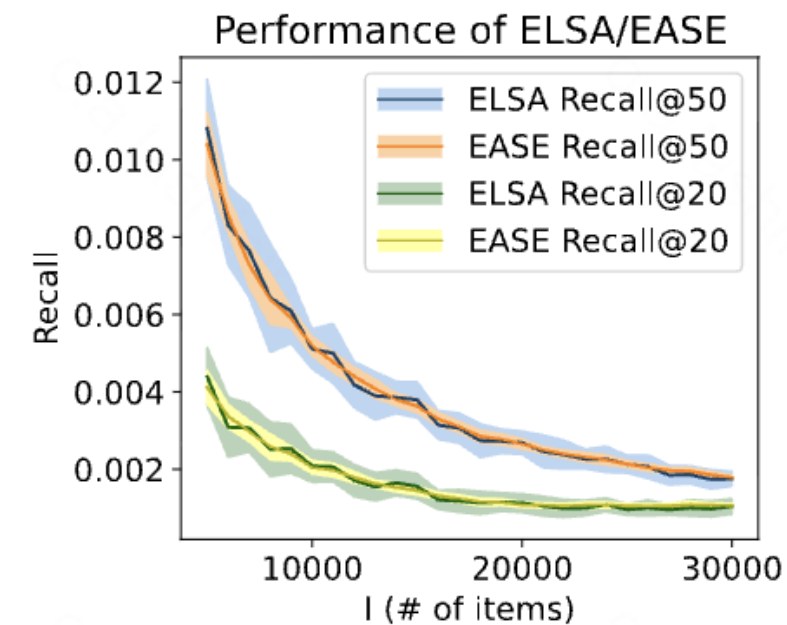
	MovieLens20M			Netflix Prize			Goodbooks-10k		
	$N@100$	$R@20$	$R@50$	$N@100$	$R@20$	$R@50$	$N@100$	$R@20$	$R@50$
Top popular	0.194	0.165	0.240	0.159	0.117	0.175	0.138	0.079	0.135
KNN	0.392	0.361	0.487	0.336	0.305	0.382	0.355	0.227	0.340
MF	0.396	0.366	0.501	0.363	0.330	0.416	0.363	0.237	0.343
EASE	0.425	0.394	0.519	0.391	0.360	0.442	0.495	0.350	0.491
ELSA	0.426	0.399	0.530	0.393	0.362	0.445	0.492	0.345	0.489



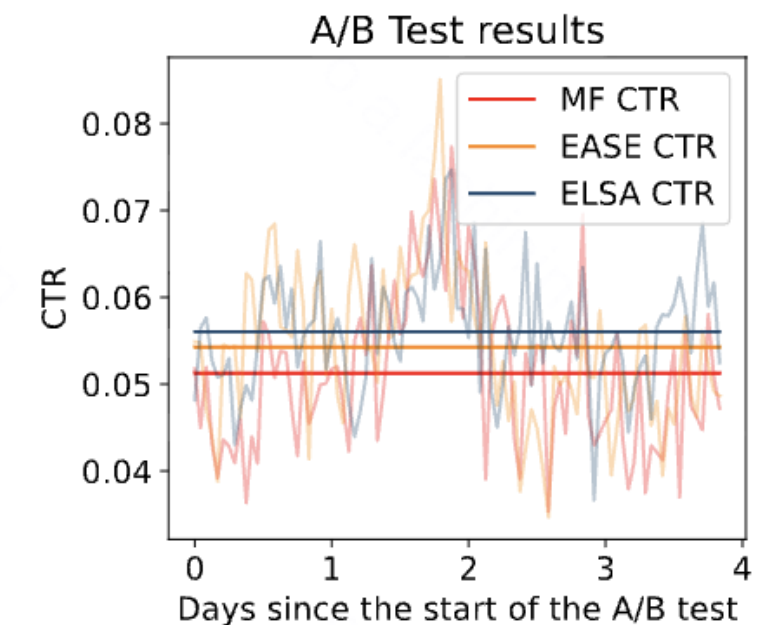
(a)



(b)



(c)



(d)

Neural Collaborative filtering

Neural collaborative filtering

[\[PDF\] acm.org](#)

[X He](#), [L Liao](#), [H Zhang](#), [L Nie](#), [X Hu](#)... - Proceedings of the 26th ..., 2017 - dl.acm.org

... Since this work focuses on the pure **collaborative filtering** setting, we use only the identity of ... into a multi-layer **neural** architecture, which we term as **neural collaborative filtering** layers, to ...

☆ Save  Cite Cited by 9229 Related articles All 16 versions

Очень много цитирований по меркам RecSys => давайте разберем

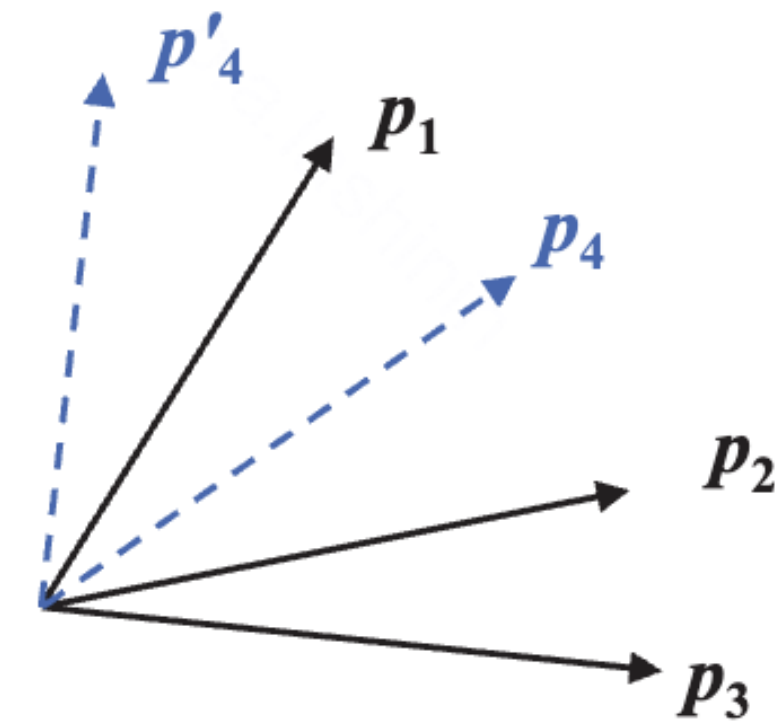
Neural Collaborative filtering

	i_1	i_2	i_3	i_4	i_5
u_1	1	1	1	0	1
u_2	0	1	1	0	0
u_3	0	1	1	1	0
u_4	1	0	1	1	1

← items →

↑ users

(a) user-item matrix



(b) user latent space

Figure 1: An example illustrates MF's limitation. From data matrix (a), u_4 is most similar to u_1 , followed by u_3 , and lastly u_2 . However in the latent space (b), placing p_4 closest to p_1 makes p_4 closer to p_2 than p_3 , incurring a large ranking loss.

Варианты размещения Юзеров в пространстве неоптимальны

Neural Collaborative filtering

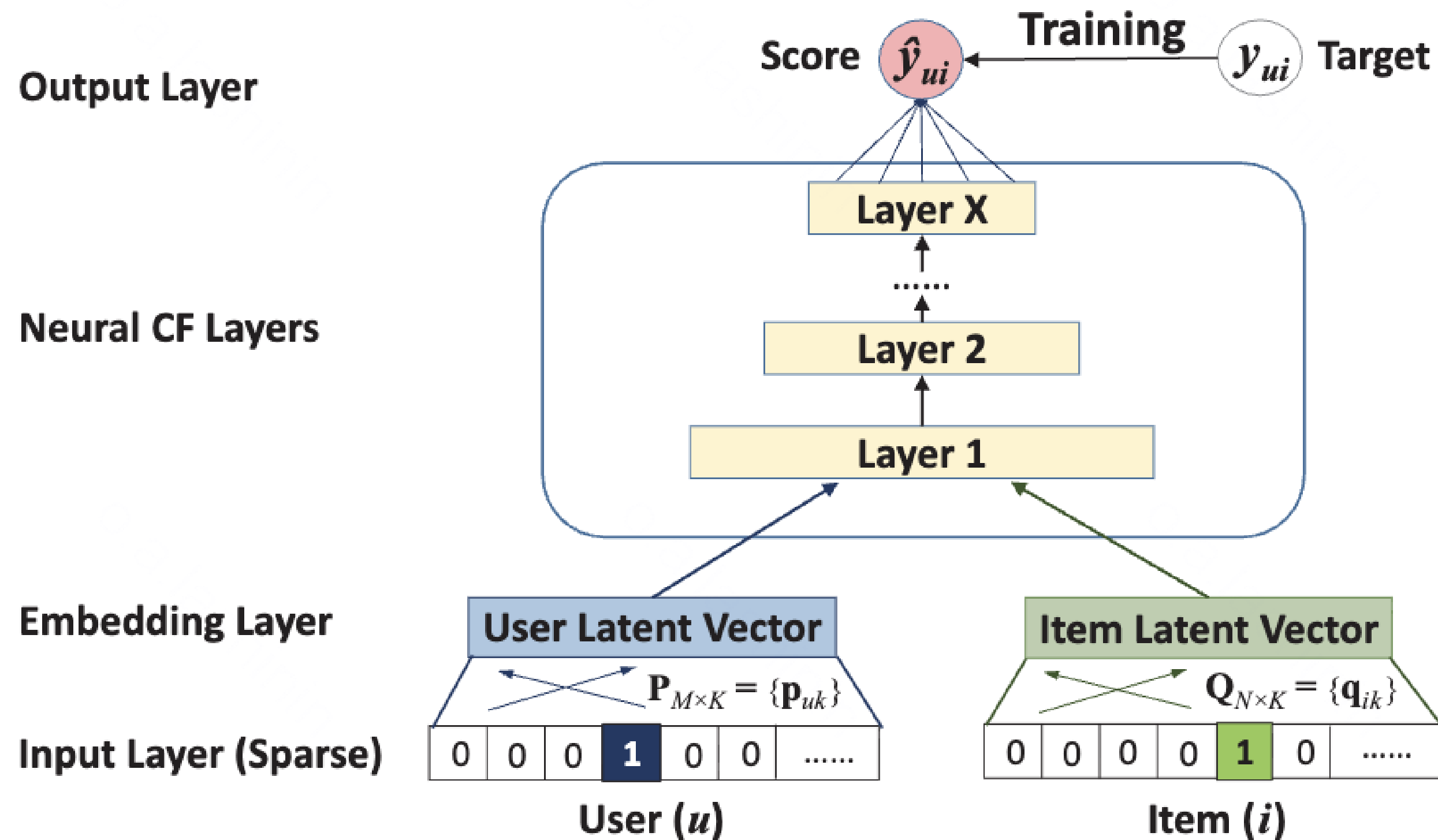


Figure 2: Neural collaborative filtering framework

Neural Collaborative filtering

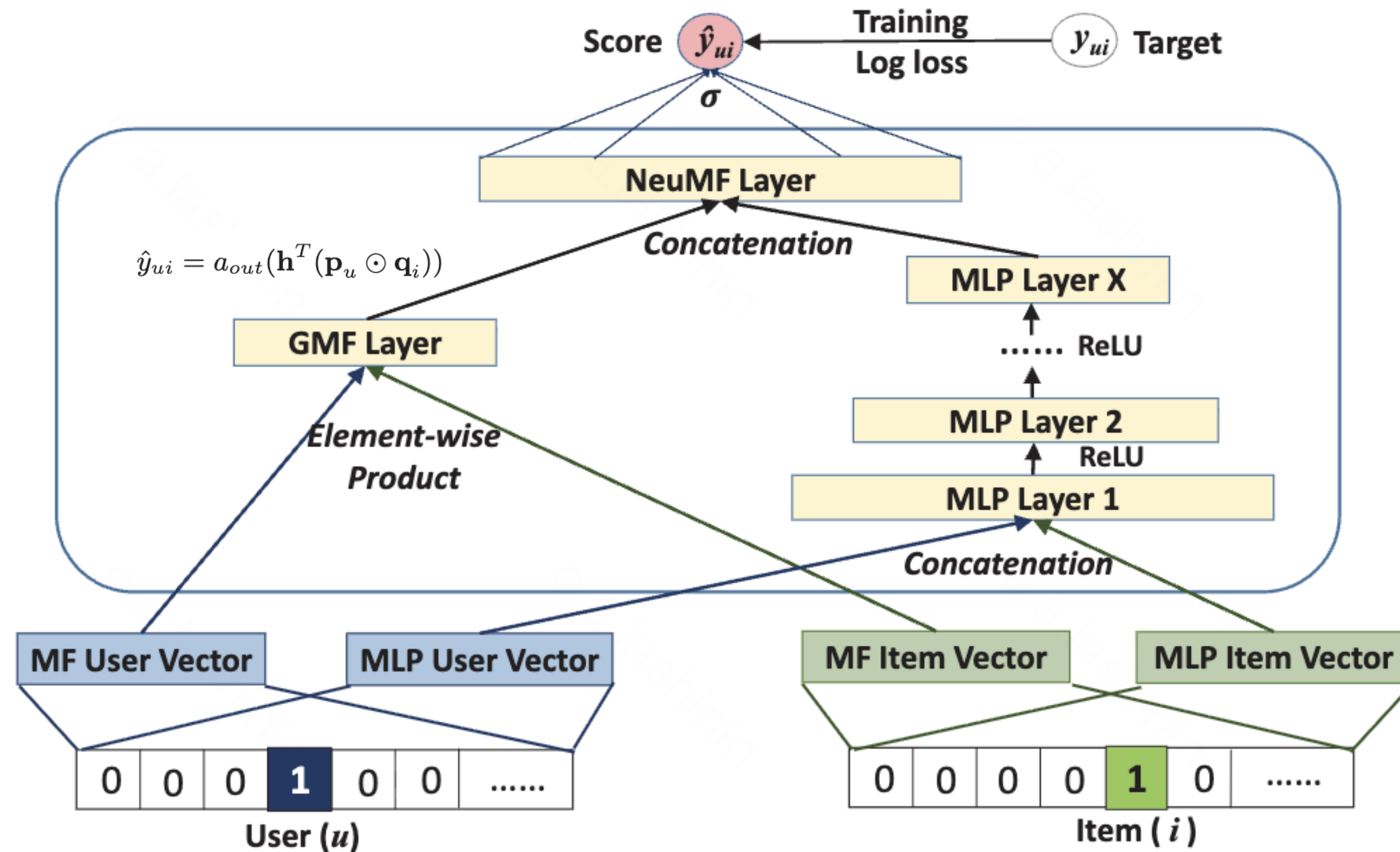


Figure 3: Neural matrix factorization model

Neural Collaborative filtering

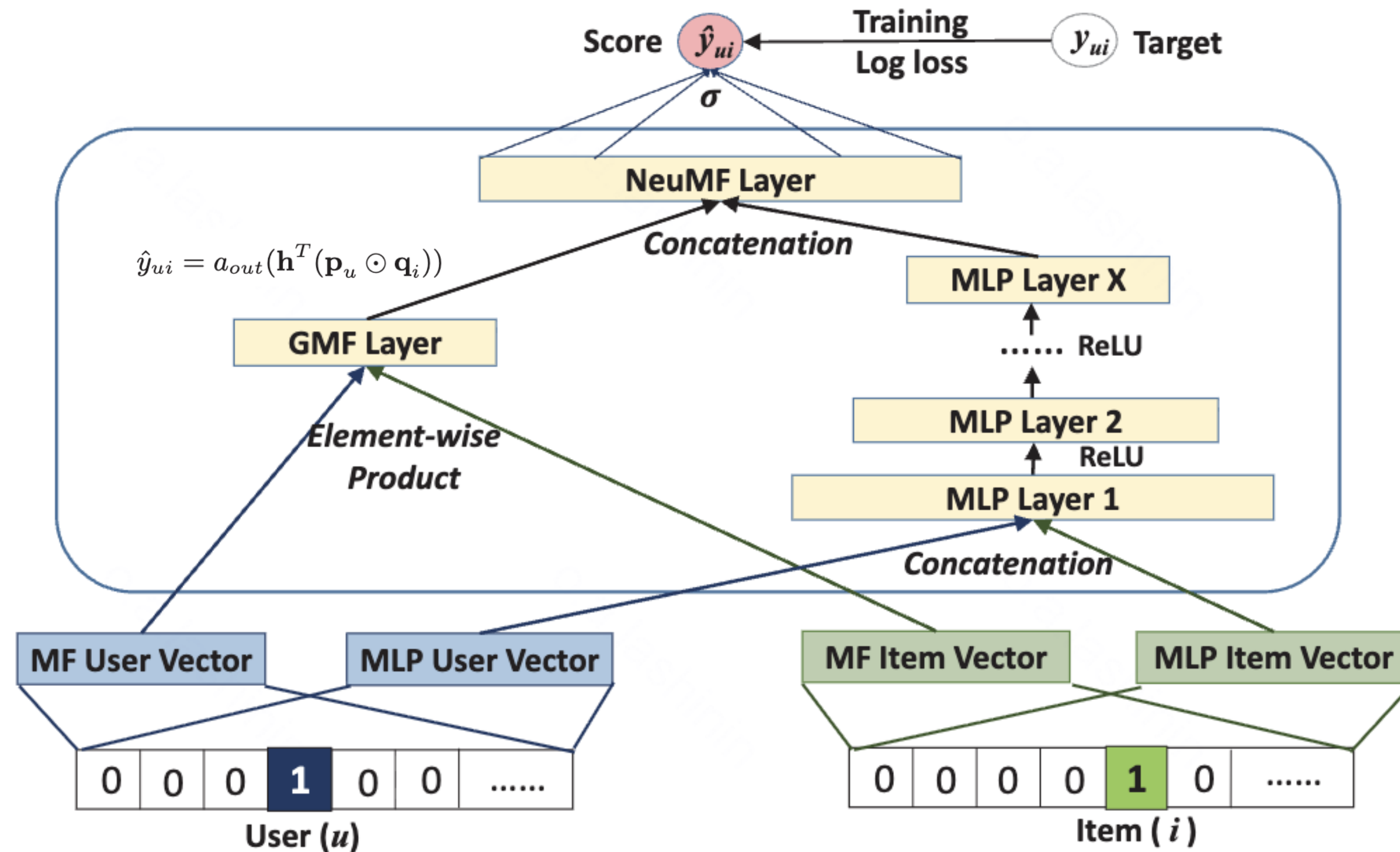


Figure 3: Neural matrix factorization model

Neural Collaborative filtering

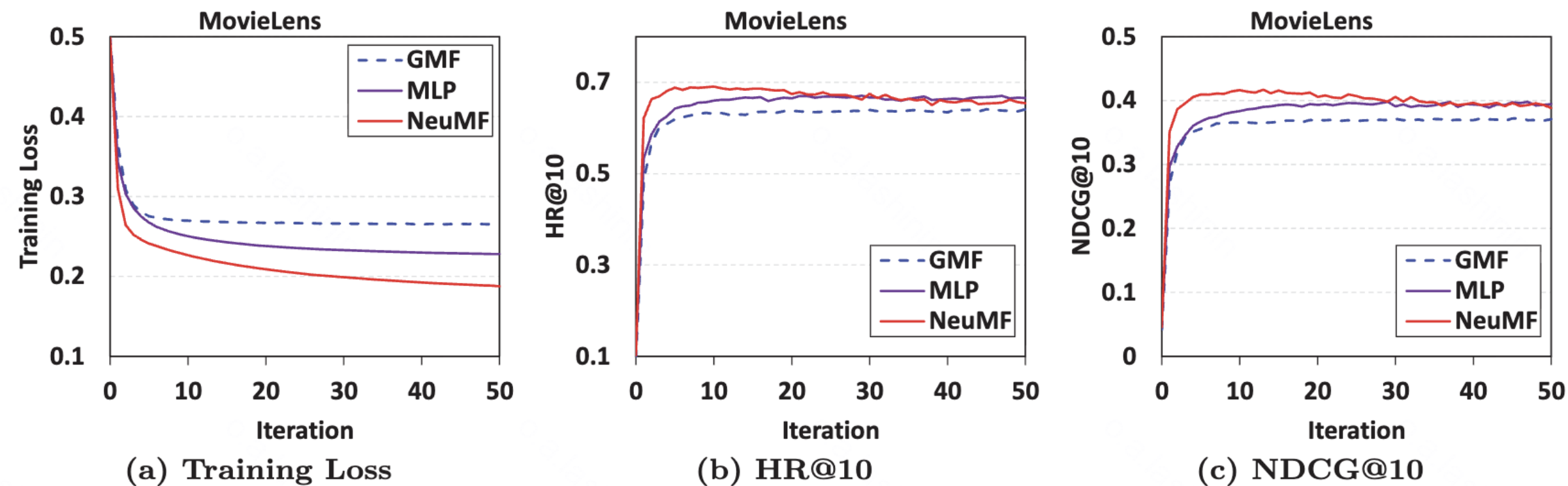
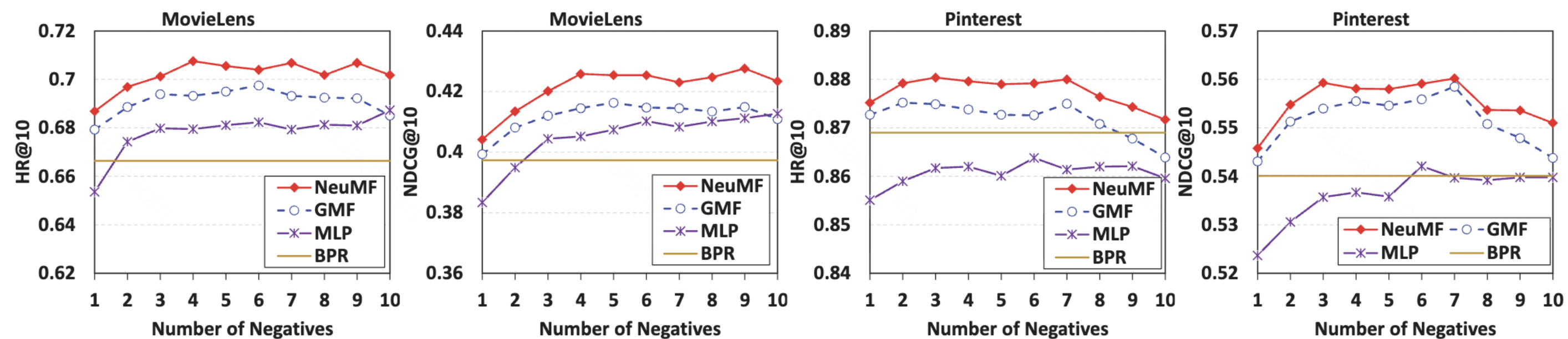


Figure 6: Training loss and recommendation performance of NCF methods *w.r.t.* the number of iterations on MovieLens (factors=8).



(a) MovieLens — HR@10 (b) MovieLens — NDCG@10 (c) Pinterest — HR@10 (d) Pinterest — NDCG@10
Figure 7: Performance of NCF methods *w.r.t.* the number of negative samples per positive instance (factors=16). The performance of BPR is also shown, which samples only one negative instance to pair with a positive instance for learning.

Neural Collaborative filtering

Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches

Maurizio Ferrari Dacrema
Politecnico di Milano, Italy
maurizio.ferrari@polimi.it

Paolo Cremonesi
Politecnico di Milano, Italy
paolo.cremonesi@polimi.it

Dietmar Jannach
University of Klagenfurt, Austria
dietmar.jannach@aau.at

Одна из немногих моделей,
которая почти не
проиграла бейзлайнам

Table 6: Experimental results for NCF.

	Pinterest			
	HR@5	NDCG@5	HR@10	NDCG@10
TopPopular	0.1663	0.1065	0.2744	0.1412
UserKNN	0.7001	0.5033	0.8610	0.5557
ItemKNN	0.7100	0.5092	0.8744	0.5629
$P^3\alpha$	0.7008	0.5018	0.8667	0.5559
$RP^3\beta$	0.7105	0.5116	0.8740	0.5650
NeuMF	0.7024	0.4983	0.8719	0.5536
	Movielens 1M			
	HR@5	NDCG@5	HR@10	NDCG@10
TopPopular	0.3043	0.2062	0.4531	0.2542
UserKNN	0.4916	0.3328	0.6705	0.3908
ItemKNN	0.4829	0.3328	0.6596	0.3900
$P^3\alpha$	0.4811	0.3331	0.6464	0.3867
$RP^3\beta$	0.4922	0.3409	0.6715	0.3991
NeuMF	0.5486	0.3840	0.7120	0.4369
SLIM	0.5589	0.3961	0.7161	0.4470

SVD

Singular value decomposition

Article Talk

Read Edit View history Tools 24 languages

From Wikipedia, the free encyclopedia

In **linear algebra**, the **singular value decomposition** (**SVD**) is a **factorization** of a **real** or **complex matrix** into a rotation, followed by a rescaling followed by another rotation. It generalizes the **eigendecomposition** of a square **normal matrix** with an orthonormal eigenbasis to any $m \times n$ matrix. It is related to the **polar decomposition**.

Specifically, the singular value decomposition of an $m \times n$ complex matrix **M** is a factorization of the form $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$, where **U** is an $m \times m$ complex **unitary matrix**, **Σ** is an $m \times n$ **rectangular diagonal matrix** with non-negative real numbers on the diagonal, **V** is an $n \times n$ complex unitary matrix, and **V**^{*} is the **conjugate transpose** of **V**. Such decomposition always exists for any complex matrix. If **M** is real, then **U** and **V** can be guaranteed to be real **orthogonal** matrices; in such contexts, the SVD is often denoted $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.

The diagonal entries $\sigma_i = \Sigma_{ii}$ of **Σ** are uniquely determined by **M** and are known as the **singular values** of **M**. The number of non-zero singular values is equal to the **rank** of **M**. The columns of **U** and the columns of **V** are called left-singular vectors and right-singular vectors of **M**, respectively. They form two sets of **orthonormal bases** $\mathbf{u}_1, \dots, \mathbf{u}_m$ and $\mathbf{v}_1, \dots, \mathbf{v}_n$, and if they are sorted so that the singular values σ_i with value zero are all in the highest-numbered columns (or rows), the singular value decomposition can be written as

$$\mathbf{M} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^*,$$

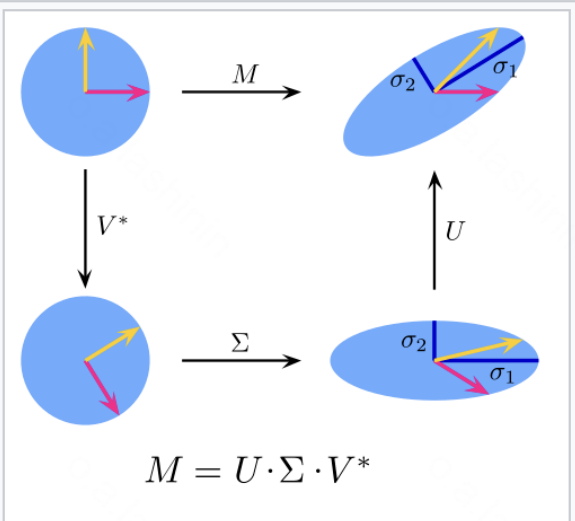


Illustration of the singular value decomposition $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$ of a real 2×2 matrix **M**.
Top: The action of **M**, indicated by its effect on the unit disc *D* and the two canonical unit vectors e_1 and e_2 .
Left: The action of **V**^{*}, a rotation, on *D*, e_1 , and e_2 .
Bottom: The action of **Σ**, a scaling by the singular values σ_1 horizontally and σ_2 vertically.
Right: The action of **U**, another rotation.

$$\begin{matrix} \text{3x3 grid} & \text{4x4 grid with colored columns} & \text{4x4 grid with colored squares} & \text{3x3 grid with colored rows} \\ \mathbf{M} & = & \mathbf{U} & \mathbf{\Sigma} & \mathbf{V}^* \\ m \times n & & m \times m & m \times n & n \times n \end{matrix}$$

https://en.wikipedia.org/wiki/Singular_value_decomposition

Не будем подробно обсуждать, так как на практике SVD в recsys практически нигде не используется сейчас

BPR

BPR: **Bayesian personalized ranking** from implicit feedback

[S Rendle](#), [C Freudenthaler](#), [Z Gantner](#)... - arXiv preprint arXiv ..., 2012 - arxiv.org

... In order to complete the **Bayesian** modeling approach of the **personalized ranking** task, we introduce a general prior density $p(\Theta)$ which is a normal distribution with zero mean and ...

☆ Save [Cite](#) Cited by 9258 [Related articles](#) [All 12 versions](#) [↔](#)

Revisiting BPR: A replicability study of a common recommender system baseline [PDF] from acm.org

Authors Aleksandr Milogradskii, Oleg Lashinin, Alexander P, Marina Ananyeva, Sergey Kolesnikov

Publication date 2024/10/8

Book Proceedings of the 18th ACM Conference on Recommender Systems

Pages 267-277

BPR

Основная идея:

- **Цель:** научить модель ранжировать **предпочтения пользователя** так, чтобы **реально взаимодействованные объекты (положительные)** стояли выше **не взаимодействованных (отрицательных)**.
- **Формулировка:** для пользователя u и пары объектов i (положительный) и j (отрицательный) модель максимизирует вероятность:

$$P(i >_u j) = \sigma(\hat{x}_{ui} - \hat{x}_{uj})$$

где $\hat{x}_{ui}$ — предсказанное «score» пользователя для айтема, σ — сигмоида.

- **Loss (функция потерь):**

$$\mathcal{L} = - \sum_{(u,i,j) \in D} \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) + \lambda \|\Theta\|^2$$

- Первая часть — **лог-вероятность правильного ранжирования**
- Вторая — регуляризация параметров Θ (эмбединги пользователей и айтемов)

Ключевые моменты:

1. BPR не предсказывает конкретный рейтинг, а только **ранжирует объекты**.
2. Работает отлично на **implicit feedback** (кликах, просмотрах, покупках).
3. Обычно реализуется через **SGD** по сэмплам (u, i, j) .

1) Мы учим pairwise лосс

2) У нас выучивается $u * I$ произведение, значение которого должно правильно упорядочивать айтемы для юзера

3) Идеальный лосс приведет к тому, что все релевантные айтемы будут сверху, все нерелевантные снизу

BPR

Основная идея:

- **Цель:** научить модель ранжировать **предпочтения пользователя** так, чтобы **реально взаимодействованные объекты (положительные)** стояли выше **не взаимодействованных (отрицательных)**.
- **Формулировка:** для пользователя u и пары объектов i (положительный) и j (отрицательный) модель максимизирует вероятность:

$$P(i >_u j) = \sigma(\hat{x}_{ui} - \hat{x}_{uj})$$

где $\hat{x}_{ui}$ — предсказанное «score» пользователя для айтема, σ — сигмоида.

- **Loss (функция потерь):**

$$\mathcal{L} = - \sum_{(u,i,j) \in D} \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) + \lambda \|\Theta\|^2$$

- Первая часть — **лог-вероятность правильного ранжирования**
- Вторая — регуляризация параметров Θ (эмбединги пользователей и айтемов)

Ключевые моменты:

1. BPR не предсказывает конкретный рейтинг, а только **ранжирует объекты**.
2. Работает отлично на **implicit feedback** (кликах, просмотрах, покупках).
3. Обычно реализуется через **SGD** по сэмплам (u, i, j) .

1) Мы учим pairwise лосс

2) У нас выучивается $u * I$ произведение, значение которого должно правильно упорядочивать айтемы для юзера

3) Идеальный лосс приведет к тому, что все релевантные айтемы будут сверху, все нерелевантные снизу

Table 4: Evaluation of the BPR model in terms of NDCG@K (N@K) and Recall@K (R@K) using various subsets of item biases and regularization features. All models in the table were trained with the SGD optimizer and uniform negative sampling. The best value is bolded, and the second-best is underlined.

Features					N@5	N@10	N@100	R@5	R@10	R@100
Item Biases	Regularization									
	User λ_u	Positive λ_i	Negative λ_j	Shared λ						
ML-20M										
X	X	X	X	X	0.2537	0.2504	0.3510	0.1246	0.1948	0.5735
X	X	X	X	✓	0.2656	0.2627	0.3665	0.1316	0.2048	0.5938
X	✓	✓	X	X	0.2721	<u>0.2695</u>	0.3694	<u>0.1353</u>	<u>0.2100</u>	0.5940
X	✓	✓	✓	X	0.2929	0.2879	0.3883	0.1398	0.2171	0.6150
✓	X	X	X	X	0.2517	0.2469	0.3499	0.1185	0.1863	0.5771
✓	X	X	X	✓	0.2419	0.2381	0.3441	0.1180	0.1855	0.5707
✓	✓	✓	X	X	0.2565	0.2514	0.3530	0.1221	0.1906	0.5766
✓	✓	✓	✓	X	<u>0.2748</u>	0.2686	<u>0.3723</u>	0.1291	0.2020	<u>0.5996</u>
MSD										
X	X	X	X	X	0.1938	0.1906	0.2753	0.1007	0.1543	0.4145
X	X	X	X	✓	<u>0.2183</u>	<u>0.2150</u>	<u>0.3115</u>	<u>0.1098</u>	<u>0.1707</u>	0.4695
X	✓	✓	X	X	0.2085	0.2057	0.3018	0.1052	0.1638	0.4593
X	✓	✓	✓	X	0.1844	0.1828	0.2796	0.0939	0.1472	0.4396
✓	X	X	X	X	0.1859	0.1828	0.2706	0.0960	0.1477	0.4142
✓	X	X	X	✓	0.2249	0.2196	0.3132	0.1128	0.1734	<u>0.4659</u>
✓	✓	✓	X	X	0.2126	0.2092	0.3027	0.1066	0.1657	0.4559
✓	✓	✓	✓	X	0.1796	0.1773	0.2748	0.0891	0.1411	0.4360

Table 5: Evaluation of the BPR model in terms of NDCG@K (N@K) and Recall@K (R@K) using various subsets of optimizer and negative sampling features. For ML-20M, the models were trained without item biases and with three separate regularization factors. For MSD, the models were trained without item biases and shared regularization. The best value is bolded, and the second-best is underlined.

Features						N@5 N@10 N@100 R@5 R@10 R@100					
Optimizer				Negative Sampling							
SGD	Momentum SGD	RMSProp	Adam	Uniform	Adaptive						
ML-20M											
✓	✗	✗	✗	✓	✗	0.2929	0.2879	0.3883	0.1398	0.2171	<u>0.6150</u>
✓	✗	✗	✗	✗	✓	<u>0.3085</u>	<u>0.3020</u>	0.4012	<u>0.1488</u>	0.2299	0.6258
✗	✓	✗	✗	✓	✗	0.2669	0.2638	0.3655	0.1275	0.2017	0.5939
✗	✗	✓	✗	✓	✗	0.1379	0.1393	0.2209	0.0641	0.1086	0.4030
✗	✗	✗	✓	✓	✗	0.2853	0.2811	0.3818	0.1384	0.2157	0.6090
✗	✗	✗	✓	✗	✓	0.3137	0.3042	<u>0.3986</u>	0.1510	<u>0.2281</u>	0.6119
MSD											
✓	✗	✗	✗	✓	✗	<u>0.2249</u>	<u>0.2196</u>	<u>0.3132</u>	<u>0.1128</u>	0.1734	<u>0.4659</u>
✓	✗	✗	✗	✗	✓	0.2475	0.2395	0.3289	0.1253	0.1881	0.4730
✗	✓	✗	✗	✓	✗	0.1516	0.1518	0.2441	0.0771	0.1232	0.3966
✗	✗	✓	✗	✓	✗	0.0318	0.0329	0.0627	0.0160	0.0274	0.1161
✗	✗	✗	✓	✓	✗	0.2014	0.2000	0.2950	0.1052	0.1625	0.4484
✗	✗	✗	✓	✗	✓	0.2118	0.2088	0.3029	0.1106	0.1687	0.4537

In this section, we focus on optimizers utilizing uniform-based negative sampling exclusively. Additionally, although the model

BPR

Table 6: Evaluation results of BPR implementations, baselines, and our BPR implementation in terms of NDCG@K (N@K) and Recall@K (R@K) using two evaluation protocols. For user-based evaluation protocol, the best models for open-source BPR implementations were selected from Figure 3. For our implementation in the user-based evaluation protocol, the best models were selected from Table 5. Values are formatted for convenience¹⁴.

User-based split similar to [51]													
	Models	ML-20M						MSD					
		N@5	N@10	N@100	R@5	R@10	R@100	N@5	N@10	N@100	R@5	R@10	R@100
Baselines	ItemPop	0.1298†	0.1275†	0.1906†	0.0580†	0.0912†	0.3298†	0.0360†	0.0349†	0.0582†	0.0176†	0.0271†	0.0986†
	EASE	0.3376†	0.3290†	0.4215†	0.1612†	0.2464†	<u>0.6371†</u>	0.3368†	0.3179†	0.3907†	0.1648†	0.2378†	0.5096†
	Mult-DAE	0.2949	0.2890†	0.3881†	0.1404†	0.2183†	0.6099†	0.2095†	0.1994†	0.2674†	0.1054†	0.1535†	0.3762†
	Mult-VAE	0.3071	<u>0.3075</u>	<u>0.4158†</u>	<u>0.1600†</u>	<u>0.2460†</u>	0.6520†	0.2221†	0.2156†	0.2973†	0.1144†	0.1716†	0.4307†
BPR Implementations	Cornac	0.2564†	0.2589†	0.3691†	0.1319†	0.2097†	0.6056†	0.2138†	0.2120†	0.3114†	0.1090†	0.1704†	0.4726
	Implicit	0.2071†	0.2052†	0.2880†	0.1093†	0.1673†	0.4745†	0.1749†	0.1737†	0.2567†	0.0929†	0.1425†	0.3898†
	LightFM	0.1295†	0.1262†	0.1843†	0.0544†	0.0869†	0.3087†	0.0362†	0.0349†	0.0575†	0.0175†	0.0268†	0.0964†
	RecBole	0.1242†	0.1213†	0.1785†	0.0513†	0.0820†	0.2973†	0.0004†	0.0009†	0.0117†	0.0002†	0.0008†	0.0328†
	Ours (SGD)	0.3085	0.3020	0.4012	0.1488	0.2299	0.6258	<u>0.2475</u>	<u>0.2395</u>	<u>0.3289</u>	<u>0.1253</u>	<u>0.1881</u>	<u>0.4730</u>
	Ours (Adam)	<u>0.3137</u>	0.3042	0.3986	0.1510	0.2281	0.6116†	0.2118†	0.2088†	0.3029†	0.1106†	0.1687†	0.4537†
Global temporal split recommended by [24, 30, 43]													
	Models	ML-20M (time-split)						Yelp					
		N@5	N@10	N@100	R@5	R@10	R@100	N@5	N@10	N@100	R@5	R@10	R@100
Baselines	ItemPop	0.0939†	0.0855†	0.0990†	0.0169†	0.0274†	0.1312†	0.0034†	0.0041†	0.0105†	0.0030†	0.0055†	0.0306†
	EASE	<u>0.1419</u>	<u>0.1332</u>	0.1698	0.0261	0.0450	0.2496	<u>0.0216†</u>	<u>0.0250†</u>	0.0527†	<u>0.0182†</u>	0.0307†	0.1387†
	Mult-DAE	0.1386	0.1308	0.1691	<u>0.0262</u>	0.0461	0.2455	0.0205†	0.0239†	0.0518†	0.0173†	0.0293†	0.1369†
	Mult-VAE	0.1343	0.1294	<u>0.1722</u>	0.0260	0.0473	0.2546	0.0184†	0.0214†	0.0468†	0.0161†	0.0269†	0.1268†
BPR Implementations	Cornac	0.1315	0.1263	0.1660	0.0256	0.0445	0.2416†	0.0208†	0.0247†	<u>0.0566†</u>	0.0176†	<u>0.0310†</u>	<u>0.1565†</u>
	Implicit	0.1140†	0.1063†	0.1315†	0.0214†	0.0370†	0.191†	0.0118†	0.0140†	0.0348†	0.0101†	0.0178†	0.1004†
	LightFM	0.0983†	0.0900†	0.1038†	0.0169†	0.0285†	0.1375†	0.0022†	0.0025†	0.0030†	0.0015†	0.0026†	0.0052†
	RecBole	0.0033†	0.0031†	0.0061†	0.0004†	0.0008†	0.0095†	0.0001†	0.0001†	0.0005†	0.0001†	0.0002†	0.0018†
	Ours (SGD)	0.1436	0.1357	0.1730	0.0277	<u>0.0471</u>	<u>0.2539</u>	0.0237	0.0276	0.0608	0.0203	0.0344	0.1658
	Ours (Adam)	0.1356	0.1277	0.1637†	0.0248	0.0431	0.2373†	0.0163†	0.0195†	0.0465†	0.0139†	0.0248†	0.1314†

¹⁴Within each column, the best value is bolded, the second-best is underlined, and † indicates a statistically significant difference ($p < 0.05$) from **Ours (SGD)** model, determined by a paired t-test with Bonferroni correction [5] for multiple comparisons.

Спасибо за
внимание